

High-Precision Autonomous Landing Systems for UAVs Using Visual Navigation

BY

FRANCESCO MARCHI

B.Sc., Management Engineering, Università degli studi di Padova, Italy, 2024

THESIS

Submitted as partial fulfillment of the requirements
for the degree of Master of Science in Electrical and Computer Engineering
in the Graduate College of the
University of Illinois at Chicago, 2026

Chicago, Illinois

Defense Committee:

Danilo Erricolo, Chair
Pranav Bhounsule, Advisor
Marcello Chiaberge, Politecnico di Torino

"E quindi uscimmo a riveder le stelle."

(Dante Alighieri, Inferno, Canto XXXIV)

*Ai miei cari che hanno sorretto i miei passi quando la strada era impervia e il cielo
oscurato, trasformando la mia fatica in orizzonte.*

Per aspera ad astra.

ACKNOWLEDGMENTS

I would like to express my deepest gratitude to Professor Pranav Bhounsule, my advisor at the University of Illinois at Chicago, and to Professor Marcello Chiaberge of the Politecnico di Torino. Together, they made this thesis possible, each in their own essential way. Professor Chiaberge gave me the opportunity to be here in the first place, supporting this double-degree experience from day one and offering his full availability on every matter, academic and personal alike. Professor Bhounsule welcomed me into his lab, guided the technical development of this work with patience and insight, and gave me the freedom to make it my own. The trust they both placed in me, and the genuine care they have shown throughout, have been a constant reference point in this journey. I also thank Professor Danilo Erricolo, chair of my committee, for his support throughout the ECE program and for the careful feedback on this manuscript. I am grateful to the University of Illinois at Chicago and to the Politecnico di Torino for sustaining this double-degree program, and to the Italian Scholarship for International Studies for the financial support that allowed me to study in Chicago.

To my parents, Elena and Gianluca, and the whole family whose love and unwavering support have made every step of this path possible. They have been my first teachers and my steadiest anchor, from my earliest days in Vicenza all the way to the final pages of this thesis. Whatever I have achieved, I owe in large measure to them.

To Camilla, thank you for standing by me through the distance of these months. Sharing this journey with you, even with an ocean in between, has made it lighter and more meaningful.

ACKNOWLEDGMENTS (Continued)

To my friends back home in Italy, the ones I have made along my journey and the ones who have always been there for a lifetime in Vicenza, for the calls, the messages, and the ability to pick up exactly where we left off every time I came back. Distance has not changed what matters.

*Come sempre, di **corto muso**, ce l'abbiamo fatta.*

Grazie a tutti.

FM

CONTRIBUTION OF AUTHORS

This thesis has been developed and submitted as the final document of a joint Master of Science double-degree TOP-UIC program between the University of Illinois at Chicago (UIC), Electrical and Computer Engineering, and the Politecnico di Torino, Mechatronic Engineering, Italy. As required by the structure of this program, a companion thesis [1] is submitted at the Politecnico di Torino, which shares the same content, results, and conclusions as the present document. Both theses have been written by the author, Francesco Marchi, and are intended to jointly fulfill the degree requirements of both institutions.

The research work presented in this thesis is the original contribution of the author, carried out under the joint supervision of Professor Pranav Bhounsule (UIC, thesis advisor). Professor Danilo Erricolo (UIC) and Marcello Chiaberge (PoliTo) served as committee chair and external advisor. All experimental work, software development, simulations, and analyses reported in this document were performed by the author, under the guidance of the aforementioned supervisors.

External sources, published works, and third-party material are cited throughout the text in accordance with the citation format adopted in the CITED LITERATURE section.

TABLE OF CONTENTS

<u>CHAPTER</u>	<u>PAGE</u>
1 INTRODUCTION	1
1.1 Definition and Classifications	2
1.2 Flight Physics and Control	3
1.2.1 Quadrotor Dynamics (Simplified)	5
1.2.2 Coordinate Frame Conventions	7
1.3 Overview of Autonomous UAV Landing Approaches	7
1.3.1 Multi-Sensor Fusion Systems	8
1.3.2 Vision-Only Systems	9
1.3.3 Learning-Based Approaches	10
1.4 Thesis Objectives	10
2 POSITIONING TECHNIQUES	14
2.1 Introduction	14
2.1.1 Classification of Positioning Systems	14
2.2 Global Navigation Satellite Systems	15
2.2.1 Standard GPS	16
2.2.2 Differential and RTK GPS	16
2.3 Ultra-Wideband Positioning Systems	17
2.4 Inertial Measurement Units	18
2.5 Vision-Based Positioning	19
2.5.1 Fiducial Markers for Pose Estimation	19
2.6 ArUco Marker System	21
2.6.1 Marker Detection Pipeline	21
2.6.2 Accuracy and Limitations	23
2.6.3 Multi-Scale Marker Strategy	23
2.7 Summary	23
3 PX4 AUTOPILOT AND SYSTEM ARCHITECTURE	25
3.1 System Overview	25
3.1.1 Flight Stack and Middleware	25
3.1.2 Control Stack	26
3.2 Offboard Control Interface (MAVSDK)	27
3.3 World and Body Frame Modifications	29
3.3.1 Coordinate Frames	29
3.3.2 Simulation Environment Customization	30
4 FILTERING AND SENSOR FUSION	34

TABLE OF CONTENTS (Continued)

<u>CHAPTER</u>		<u>PAGE</u>
4.1	Software Adaptation	34
4.2	Pinhole Camera Model and Pose Estimation	36
4.2.1	Pixel-to-metre ratio.	36
4.3	Visual Navigation Target Design	37
4.3.1	Nested Target Architecture	37
4.3.2	Vision Debugging Tool	39
4.4	State Estimation via Kalman Filter	40
4.4.1	Asynchronous Communication and Zero-Order Hold	41
5	SYSTEM CONTROL ALGORITHM	43
5.1	System State Machine	43
5.2	PID Control Logic and Geometric Correction	44
5.2.1	Integral action and anti-windup	47
5.2.2	Feed-Forward Factor	48
5.2.3	Gain Scheduling and Dynamic Alignment Threshold	49
5.3	Landing Phase	50
5.3.1	Descent Logic and Land Command	51
5.3.2	Termination Sequence (Touchdown)	51
6	SIMULATION	53
6.1	Estimator Tuning and Covariance Matrices	53
6.1.1	Multi-Scale Detection Logic	53
6.2	Kalman Filter Sensitivity Analysis	55
6.2.1	Effect of Q	55
6.2.2	Effect of R	57
6.2.3	Tracking Error vs. Altitude	59
6.3	Overall Simulation With a Still Target	59
6.4	Overall Simulation of Landing on a 1 km/h Moving Platform	62
6.4.1	Representative Mission Profile	63
6.4.2	Aggregate Touchdown Accuracy	65
6.4.3	Image-Plane Tracking Error	68
6.4.4	Control Outputs	70
6.4.5	Control Effort Distribution	71
6.4.6	Summary of Moving-Platform Simulation Results	72
6.5	Lost-Target Recovery and Search Behaviour	74
7	EXPERIMENTAL ANALYSIS	76
7.1	Sim-to-Real Transfer and Deployment Strategy	76
7.2	Drone Setup	77
7.3	Rover Setup	78
7.4	Camera	80
7.4.1	Camera Calibration and Environmental Factors	81

TABLE OF CONTENTS (Continued)

<u>CHAPTER</u>		<u>PAGE</u>
7.5	Touchdown and Disarming Sequence	82
7.6	Results	82
7.6.1	Landing Trajectory Analysis	83
7.6.2	Attitude and Vertical Velocity Tracking	86
7.6.3	Vibration Analysis	89
7.6.4	Discussion: Transferability of Simulation Results	91
8	CONCLUSION	94
8.1	Summary of Contributions	95
8.2	Limitations	96
8.3	Future Work	98
8.3.1	Near-Term Improvements	98
	APPENDICES	100
	APPENDIX A	101
	APPENDIX B	105
	CITED LITERATURE	109
	VITA	112

LIST OF TABLES

<u>TABLE</u>		<u>PAGE</u>
I	COMPARISON OF AUTONOMOUS LANDING SYSTEMS.	11
II	EXAMPLE OF ARUCO DICTIONARIES AND QUALITATIVE PROPERTIES [2].	22
III	COMPARISON OF POSITIONING TECHNIQUES FOR UAV LANDING	24
IV	PHYSICAL DIMENSIONS OF THE NESTED ARUCO LANDING TARGET.	39
V	MAIN PARAMETERS OF THE VISION BASED LANDING CONTROLLER.	54
VI	SENSITIVITY OF LANDING ACCURACY TO THE PROCESS NOISE PARAMETER Q.	56
VII	SENSITIVITY OF LANDING ACCURACY TO THE MEASUREMENT NOISE PARAMETER R.	58
VIII	AGGREGATE TOUCHDOWN ACCURACY OVER 13 SIMULATED LANDINGS ON A PLATFORM MOVING AT 1KM/H.	67

LIST OF FIGURES

<u>FIGURE</u>		<u>PAGE</u>
1	Phantom quadcopters	2
2	The six degrees of freedom three dimensions and three rotations.	4
3	Schematic illustration of GNSS positioning	15
4	UWB positioning method	18
5	Generated ArUco markers for UAV landing	21
6	Schematic view of the PX4 cascaded control architecture and the inser- tion point of the proposed visual landing controller.	27
7	Simulated UAV model (gz_x500_vision) on the nested ArUco landing pad in Gazebo Harmonic.	30
8	Top view of the coloured landing pad with nested ArUco markers used in simulation.	32
9	Nested target used in our research generated for the target frame using the OpenCV library in Python.	38
10	State machine of the landing algorithm.	44
11	High-level architecture of the landing controller.	47
12	Effect of process noise covariance on the Kalman filter.	57
13	Landing accuracy analysis with the right tune of R.	58
14	Image-plane tracking error magnitude as a function of altitude, illustrat- ing the optical gain explosion below 1 m.	59
15	Simulation with a stationary landing platform.	60
16	Statistical analysis of 9 consecutive simulated landings on a still platform.	62
17	Representative simulated landing on a moving platform: altitude profile (top), velocity tracking with $\text{RMSE}_{\text{tot}} = 0.284 \text{ m/s}$ (centre), and offboard control outputs (bottom).	63
18	Statistical analysis of 13 consecutive simulated landings on a platform moving at 1 km/h. Left: corrected CoM touchdown dispersion with $\text{CEP}_{50} = 2.33 \text{ cm}$ (dashed circle). Right: boxplot distribution of lateral and longitudinal errors.	66
19	Pixel error convergence in the x -axis (e_u) for 13 consecutive simulated landings on a moving platform. The grey horizontal line denotes the target setpoint ($e_u = 0$). Distinct line styles and grey shades are used to facilitate identification in printed form.	68
20	Image-plane tracking error magnitude as a function of altitude for 13 simulated landings. The red dashed curve is the dynamic alignment threshold $\varepsilon(h) = \varepsilon_0 \cdot \kappa(h)$	70

LIST OF FIGURES (Continued)

<u>FIGURE</u>		<u>PAGE</u>
21	Control effort distribution during the final approach ($h < 1.5$ m) for 13 simulated landings on a moving platform. Left: pitch (forward) velocity commands ($\sigma_{v_x} = 0.0378$ m/s). Right: roll (lateral) velocity commands ($\sigma_{v_y} = 0.0326$ m/s).	72
22	Simulation with a moving platform and performing a saving hovering to regain the target.	74
23	Holybro x500 V2 used.	77
24	Husky robot used outdoor with the landing site over it.	79
25	Camera mounted on the body frame of the drone.	80
26	Progression of the UAV landing sequence. (a)-(b) Aerial detection and alignment phase. (c)-(f) Final descent and landing on the mobile platform.	83
27	Representative outdoor landing on the moving platform: altitude profile (top), velocity tracking with $RMSE_{tot} = 0.284$ m/s (centre), and offboard control outputs (bottom). Conditions: 15, wind 14km/h SSE.	85
28	Vertical velocity tracking (top), roll attitude tracking (centre), and pitch attitude tracking (bottom) during the outdoor landing. Dashed lines denote commanded setpoints; solid lines denote actual measured values.	87
29	Accelerometer time history (top) and FFT frequency spectrum (bottom) recorded during the outdoor landing. Sampling frequency ≈ 191 Hz.	90

LIST OF ABBREVIATIONS

<i>UAV</i>	Unmanned Aerial Vehicle
<i>UGV</i>	Unmanned Ground Vehicle
<i>GVA</i>	Ground Vehicle Asset
<i>PID</i>	Proportional-Integral-Derivative
<i>KF</i>	Kalman Filter
<i>CEP</i>	Circular Error Probable
<i>RMSE</i>	Root Mean Square Error
<i>SITL</i>	Software In The Loop
<i>MAVSDK</i>	MAVLink Software Development Kit
<i>FoV</i>	Field of View
<i>PnP</i>	Perspective-n-Point
<i>NED</i>	North-East-Down
<i>FRD</i>	Forward-Right-Down
<i>IMU</i>	Inertial Measurement Unit
<i>GNSS</i>	Global Navigation Satellite System
<i>RTK</i>	Real Time Kinematic
<i>UWB</i>	Ultra-Wideband

LIST OF ABBREVIATIONS (Continued)

$\mathcal{F}_c$	Camera reference frame
e_x, e_y	Tracking error in the image plane (pixels)
x, y, z	Position coordinates of the UAV
$\hat{x}, \hat{y}$	Kalman-filtered marker centre coordinates
h	Relative altitude of the UAV
$\dot{\hat{x}}, \dot{\hat{y}}$	Estimated marker pixel velocities
v_x, v_y, v_z	Body-frame velocity commands sent to PX4
ϕ, θ, ψ	Roll, pitch, and yaw Euler angles
$\mathbf{x}_k$	Discrete-time state vector at time step k
Δt	Sampling time
$\mathbf{F}$	State transition matrix
$\mathbf{H}$	Measurement matrix
$\mathbf{Q}$	Process noise covariance matrix
$\mathbf{R}$	Measurement noise covariance matrix
K_p, K_d, K_i	Proportional, derivative, and integral gains
K_{ff}	Feed-forward gain
$\gamma(h)$	Altitude-dependent dynamic damping factor
$\kappa(h)$	Dynamic alignment cone multiplier

LIST OF ABBREVIATIONS (Continued)

ε_0	Nominal alignment threshold(pixels)
$\mathbf{K}$	Camera intrinsic matrix
$L_{\text{out}}, L_{\text{in}}$	Outer / inner ArUco marker side length
f	Focal length (pixels)
O_y, O_z	Physical camera offsets relative to the center of mass
$\mathbf{K}_k$	Kalman gain at step k
$\mathbf{I}_n$	Identity matrix of size n

SUMMARY

This thesis addresses the problem of autonomous precision approaching and landing of a small quadrotor UAV on a moving ground vehicle using only onboard monocular vision. A nested ArUco fiducial marker target is designed to guarantee detection from cruise altitude down to the final centimetres of the descent. The relative position and velocity of the target are estimated by a linear Kalman filter operating in the image plane, and a gain-scheduled PID controller with spatial feed-forward compensation generates body-frame velocity commands that are injected into the PX4 autopilot via MAVSDK offboard mode.

The system is validated both in Gazebo Harmonic Software-In-The-Loop simulation and in outdoor experiments with a Holybro x500 V2 quadrotor landing on a Husky rover moving at up to 1 km/h. Simulation results over 14 trials yield a Circular Error Probable $CEP_{50} = 2.5\text{ cm}$ and a total Root Mean Square Error of 2.75 cm . Real-world flights confirm the viability of the sim-to-real transfer, with the same control gains producing stable landings without re-tuning.

CHAPTER 1

INTRODUCTION

Up until a decade ago, the robotics community mostly regarded multirotor control as an academic benchmark on attitude stabilisation and dynamic modelling, as exemplified by the foundational work of Mahony et al. in 2012 [3]. Today, however, the engineering narrative has passed: drones are no longer confined within laboratory boundaries, but are really implemented in complex missions such as infrastructure inspection and agile logistics.

A particularly application study are the missions that involves operations in GNSS-denied environments such as urban canyons, port areas, ship decks and offshore platforms [4]. It is clear that, in these contexts, satellite-based navigation may provide sufficient accuracy for en-route guidance, but it is generally inadequate to guarantee a safe touchdown on small targets whose characteristic dimensions are on the order of a few tens of centimetres [4]. This technological gap between navigation accuracy and landing accuracy motivates the adoption of local, high-resolution sensing modalities and dedicated landing aids, especially for small multirotor UAVs [4].

For instance, Eling et al. [4] showed that Differential GPS and Real-Time Kinematic (RTK) corrections can reduce navigation errors to the centimetre level, at the cost of deploying base stations nearby and maintaining reliable communication links [4]. A similar approach, Ultra-Wideband (UWB) ranging systems provide sub-decimetre accuracy but require pre-calibrated fixed anchors around the landing area [5]. Complementary research has explored modular ROS2-

based architectures for precision auto-landing of UAVs on moving ground vehicles using visual tracking and feedforward compensation [6], further highlighting the need for accurate local sensing and robust control strategies.



Figure 1: "Phantom quadcopters - recreational and commercial UAVs for aerial cinematography and photography" by Nagarjun is licensed under CC BY 4.0.

1.1 Definition and Classifications

UAVs are classified according to several criteria, which mostly consist in lifting mechanism, size, endurance and intended application. From a lift-generation standpoint, two main categories are commonly distinguished by the scientific world:

- **Fixed-wing:** Similar to traditional airplanes, they use aerodynamic surfaces to generate lift and are well-suited for long-range, energy-efficient missions. Their main limitation

is the need for runways or catapult systems for take-off and landing, which complicates operations in constrained environments [3].

- **Multirotors:** Such as quadcopters (the `gz_x500_vision` model used in this work), hexacopters or octocopters. They rely on multiple rotors to provide both lift and propulsion, and their ability to hover and perform vertical take-off and landing (VTOL) makes them ideal for inspection, close-proximity interaction and high-precision tasks [3].

Within the multirotor category, UAVs cover a wide range of sizes, from micro and mini platforms (for indoor or short-range outdoor operations) to larger vehicles (capable of carrying heavy payloads over extended distances) [7]. Small multirotors in the 2-5 kg class are particularly attractive for precision-landing research, as they combine agility, relatively low kinetic energy and the ability to operate from compact, possibly moving platforms such as ground robots, vehicles or ship decks [8].

1.2 Flight Physics and Control

A quadrotor is a rigid body evolving in three-dimensional space and is therefore characterized by six degrees of freedom (DOF): three of them are translational motions along the inertial axes x , y , and z , and the other three are rotational motions about them, commonly referred to as roll (ϕ), pitch (θ), and yaw (ψ).

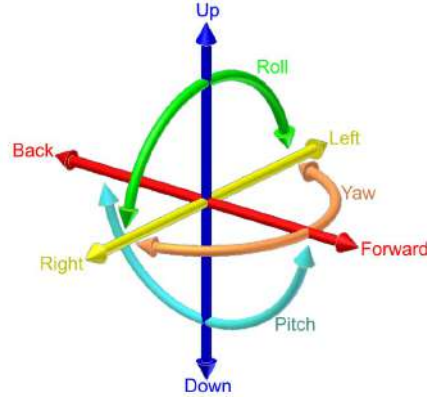


Figure 2: The six degrees of freedom three dimensions and three rotations.

Despite having six degrees of freedom, the system is actuated by only four independent control inputs, corresponding to the angular velocities of the four rotors. As a result of this, the drone is an underactuated system in which inputs generate a collective thrust force and three control torques, so not all the degree of freedom can be directly controlled. In manual operation, these motions are commanded directly by a human pilot and are easily achievable; however, in autonomous systems, they are regulated by a flight stack (PX4 in this work). This PX4 fuses data from inertial measurement units, GNSS receivers and other onboard sensors to maintain stability and to track high-level position or velocity setpoints [3].

As is often the case in engineering, the real model could be simplified by its dynamical one. As a matter of fact, a quadrotor can be easily described as a rigid body subject to gravity, thrust and aerodynamic drag [3]. The total thrust generated by the four propellers acts approximately

along the negative body-frame z -axis, while differential thrust and reaction torques create roll, pitch and yaw moments [3]. The flight controller shapes the movement of the center of mass (COM) and the torque rate on the 3 axes of the model [3].

As we will analyze in Chapter 3, these dynamics are handled by a cascaded control architecture personalized, ad hoc, for our focus [9]. An inner attitude and rate loop tracks desired angular rates and orientations at high bandwidth (also active as a low pass filter), ensuring that the vehicle remains stable even under disturbances and modeling errors; on the other hand, an outer position or velocity loop operates at a lower frequency, converting position errors in the navigation frame into attitude and thrust setpoints [9].

1.2.1 Quadrotor Dynamics (Simplified)

As previously mentioned, the drone can be modelled as a rigid body subject to thrust and aerodynamic drag [3]. In the vertical direction, the dominant dynamics can be approximated by [3]

$$m\ddot{z} = T - mg, \quad (1.1)$$

where m is the vehicle mass, T is the total thrust generated by the four propellers and g is the gravitational acceleration. Using the motor speeds, the flight controller adjusts T to track altitude commands.

Lateral motion in the horizontal plane is obtained by tilting the vehicle. Under small-angle assumptions [3], the accelerations along the body-frame x and y axes can be written as

$$m\ddot{x} \approx T\theta, \quad m\ddot{y} \approx -T\phi, \quad (1.2)$$

where ϕ and θ denote roll and pitch angles, respectively [3]. In this way, the position controller does not act directly on $\ddot{x}$ and $\ddot{y}$, but instead commands suitable attitude setpoints that generate the desired horizontal accelerations through the tilt of the thrust vector [9].

Equation 1.1 and Equation 1.2 are obtained under standard simplifying assumptions (rigid-body dynamics) small attitude angles and negligible lateral aerodynamic drag, which are commonly used for a modelization problem and simulation [3]. Although these assumptions do not capture all nonlinear effects, they provide an accurate qualitative description of vehicle behaviour in the operating regime considered in this work, where the UAV performs moderate-speed manoeuvres around a hovering condition [3]. In particular, they clearly show that horizontal accelerations are generated indirectly through attitude changes, which is reflected in the structure of the cascaded controllers used in modern flight stacks such as PX4 [9].

Moreover, Equation 1.1 and Equation 1.2 highlight a fundamental aspect of precision landing control and the primary challenge it presents: even small errors in the estimated lateral position or delays in the control loop translate into undesired attitude oscillations when the controller attempts to correct the perceived error [3]. This effect becomes particularly detrimental and critical during the final approach, when a phenomenon known as optical gain explosion [2] makes the effective pixel-to-metre ratio of the camera increase rapidly and overshoot the controller response. To mitigate this, the proposed system combines a carefully tuned vision-based controller with explicit latency compensation via a Kalman filter and a dynamic alignment threshold strategy, as detailed in Chapter 4 [2].

1.2.2 Coordinate Frame Conventions

The system involves three principal reference frames whose relationships must be carefully managed:

- (i) **Inertial frame $\mathcal{W}$:** PX4 uses the NED (North-East-Down) convention; MAVSDK body-frame commands are expressed in FRD (Forward-Right-Down).
- (ii) **Body frame $\mathcal{B}$:** attached to the UAV centre of mass, oriented FRD.
- (iii) **Camera frame $\mathcal{C}$:** the camera is mounted downward-facing; its z -axis points along the optical axis (toward the ground), and the image axes (u, v) correspond to the camera x and y axes.

The mapping between the image-plane error (e_x, e_y) in pixels and the body-frame velocity commands (v_x, v_y) requires a sign inversion and axis swap due to the camera mounting orientation. Specifically, the vertical pixel error e_y drives the pitch velocity v_x , while the horizontal pixel error e_x drives the roll velocity v_y , as formalised in Equation 5.1 and Equation 5.2 of Chapter 5.

1.3 Overview of Autonomous UAV Landing Approaches

Autonomous landing of UAVs on moving platforms has been an active research topic for over a decade, driven by applications ranging from package delivery and mobile recharging stations to maritime and military operations. Existing approaches can be broadly categorized into three families based on the primary sensing modality: multi-sensor fusion systems, vision-only systems, and learning-based methods.

1.3.1 Multi-Sensor Fusion Systems

The most common approach in the literature combines multiple positioning technologies to guarantee robustness across different relative distances. Rigogliuso [10] implemented a three-state estimation machine that transitions between GPS, Ultra-Wideband (UWB) and camera-based measurements depending on the drone-rover separation, through a 15-state Kalman filter fuses all sensor streams and a PID velocity controller with predictive compensation enables landings at speeds above 30km/h in simulation. However, as stated the system requires four UWB anchors on the rover, an RTK-GPS base station and a compass, resulting in significant infrastructure overhead.

Lee et al. [11] proposed a cooperative multi-sensor architecture combining differential GPS with infrared beacons for shipboard recovery of a fixed-wing UAV. While achieving sub-metre accuracy, again the approach depends heavily on external infrastructure that may not be available in field conditions.

Kyristsis et al. [12] employed a fusion of RTK-GPS and monocular vision for landing on a moving platform, demonstrating that vision can correct residual GPS drift in the final approach. Once again, the limitation remains the need for RTK corrections from a nearby base station.

The key trade-off in multi-sensor systems is between accuracy and deployment complexity. While they achieve excellent performance, the requirement for cooperative infrastructure (UWB anchors, RTK base stations, inter-vehicle communication links) limits their applicability to scenarios where such equipment can be pre-installed on the target.

1.3.2 Vision-Only Systems

Vision-based methods are attractive because they require no cooperative infrastructure on the landing platform beyond a passive visual target. In this area of research, Araar and Aouf [13] proposed an adaptive fuzzy controller guided by a downward-facing camera tracking an H-shaped pattern, achieving robust landing on a slowly moving vehicle. However, the system was limited to speeds below 5km/h and used a single marker scale, making it sensitive to altitude-dependent resolution changes.

Falanga et al. [14] demonstrated aggressive quadrotor manoeuvres including high-speed landing using visual-inertial odometry and AprilTag detection, achieving landings at up to 1.5m/s relative velocity. Their system fused IMU data with visual measurements in an Extended Kalman Filter, but required a high-performance onboard computer (Intel NUC) that exceeds the computational budget of lightweight platforms.

Ling et al. [15] provided a comprehensive survey of autonomous landing techniques, identifying marker detectability at varying altitudes as one of the primary challenges for monocular systems. This observation directly motivates the nested marker strategy adopted in the present work.

More recently studies focused on multi-scale fiducial targets, for example Garrido-Jurado et al. [16] introduced fractal ArUco markers that embed multiple codes at different scales within a single pattern. This idea is elegant, but their detection pipeline is computationally heavier than standard ArUco detection and has not been validated for real-time UAV landing at the frame rates required by our application.

1.3.3 Learning-Based Approaches

Deep-learning methods have been explored for target detection and control in UAV landing, as done in Polvara et al. [17], where they trained a CNN to estimate relative pose from downward-facing images, demonstrating sim-to-real transfer for stationary platform landing. Rodriguez-Ramos et al. [18] applied deep reinforcement learning to achieve autonomous landing on a moving platform in simulation. While promising, these methods typically require large training datasets, offer limited interpretability, and their robustness to domain shift in outdoor conditions remains an open question.

1.4 Thesis Objectives

Given all this above, the primary objective of this thesis is the development, implementation and validation (in simulation and real life), of a robust control system for the autonomous precision landing of a quadrotor UAV on a moving platform GVA (Ground Vehicle Asset) [1].

The main challenge of this work concerns managing the intrinsic limitations of monocular vision systems and accuracy, specifically regarding processing latency and the drastic variation in pixel density (as said above) during the final approach phase.

TABLE I: COMPARISON OF AUTONOMOUS LANDING SYSTEMS.

System	Sensor	Infra.	Max speed	CEP	Sim-to-Real
Rigogliuso	GPS+UWB+Cam	High	30km/h	$<10\text{ cm}$	Partial
Lee et al.	DGPS+IR	High	N/A	$<1\text{ m}$	Yes
Araar & Aouf	Monocular	None	5km/h	15cm	No
Falanga et al.	Stereo+IMU	None	5.4km/h	—	Yes
Polvara et al.	Monocular+DL	None	0km/h	—	Partial
This work	Monocular	None	1km/h	3.51cm	Yes

Specifically, the following objectives are pursued:

- **Development of a Multi-Scale Vision System:** Implementation of a visual tracking algorithm based on nested fiducial markers (*Nested ArUco Markers*). This approach aims to guarantee visibility at high altitudes via a large outer marker and millimeter-level precision at low altitudes ($h < 1.5m$) via a smaller inner marker, effectively resolving the problem of *field-of-view clipping* during the final descent.
- **Robust State Estimation:** Design and tuning of a Linear Kalman Filter for the estimation of position $(\hat{x}, \hat{y})$ and, crucially, the relative velocity $(\hat{v}_x, \hat{v}_y)$ of the target. The

goal is to compensate for vision system latency and provide clean signals to the controller, enabling the tracking of non-cooperative targets.

- Altitude-Scheduled Alignment Threshold:** Instead of a fixed pixel tolerance, this work implements a dynamic alignment threshold that scales with the current altitude of the vehicle. At cruise altitude ($h \geq 2.0\text{ m}$) the acceptable image-plane error is relaxed to $1.5\varepsilon_0 = 165\text{ px}$, so that the descent phase can begin as soon as the target is reliably acquired, even in the presence of moderate lateral offset. As the UAV descends, the threshold narrows linearly until it reaches the nominal value $\varepsilon_0 = 110\text{ px}$ at $h \leq 0.7\text{ m}$, where the high pixel-to-metre ratio makes each pixel a reliable proxy for a small physical distance and precise centering is critical for a safe touchdown. This strategy prevents limit-cycle behavior during the initial descent while preserving the positioning accuracy required in the final approach.
- Tracking Logic (Feed-Forward):** Integration of a *Feed-Forward* term based on the estimated velocity of the target to minimize the steady-state error during the platform's motion.
- Validation in Simulated Environment:** Experimental verification of the proposed architecture using the *Gazebo Harmonic* simulation environment and the MAVSDK framework, testing the system's robustness at various platform speeds (up to 1 km/h) and under simulated disturbances.
- Sim to Real transfer:** The key idea of the sim-to-real transfer is to treat the landing controller as a hardware-agnostic module that consumes a stream of images and vehicle

states and produces body-frame velocity commands. In SITL, images are read from a virtual downward-facing camera via Gazebo topics and the vehicle is reached through a UDP MAVLink endpoint, whereas in the real experiments both data streams are replaced by a physical USB camera and a serial MAVLink link over UART.

The expected result is a system capable of locking onto, tracking and landing on the moving platform, all this balancing precision with stability in order to ensure operational safety throughout all mission phases.

The implemented control law for the horizontal velocity $u(t)$ is defined as:

$$u(t) = K_p \cdot e(t) + K_d \cdot \dot{e}(t) + K_{ff} \cdot \hat{v}_{target} \quad (1.3)$$

where K_p, K_d, K_{ff} are the tuned PID and Feed-Forward gains, and $e(t)$ denotes the visual position error.

CHAPTER 2

POSITIONING TECHNIQUES

2.1 Introduction

The first question to be asked is: *How can accurate and reliable positioning be achieved?* As stated before, reaching an accurate and reliable positioning is a critical requirement for autonomous UAV landing on moving platforms. So, this chapter will provide a technical overview of the main positioning techniques applicable to UAV navigation, providing an outlook on vision-based methods, analyzing the advantages and limitations of each approach; motivating the adoption of a visual fiducial marker system based on ArUco markers.

2.1.1 Classification of Positioning Systems

Positioning systems for unmanned aerial vehicles can be classified according to the nature of the measurements and the reference frame in which the position is estimated; a common distinction is made between absolute and relative positioning systems [4].

For precision landing tasks, relative positioning methods are generally preferred due to their superior local accuracy.

- **Absolute Positioning Systems:** These are typically based on satellite navigation or pre-mapped environments. These solutions offer global consistency, but suffer from limited accuracy in the final landing phase where the level of accuracy drops to centimeters [4].

- **Relative Positioning Systems:** Relative positioning relies on measurements taken with respect to a nearby reference object or target. Vision-based and radio-frequency-based systems fall into this category and are particularly suitable for short-range, high-precision maneuvers.

2.2 Global Navigation Satellite Systems

The most common systems for UAV navigation are Global Navigation Satellite Systems (GNSS), most notable examples are GPS, GLONASS, and Galileo; they estimate the vehicle position by measuring the time-of-flight of radio signals transmitted by multiple satellites around the globe and performing a trilateration of these signals.

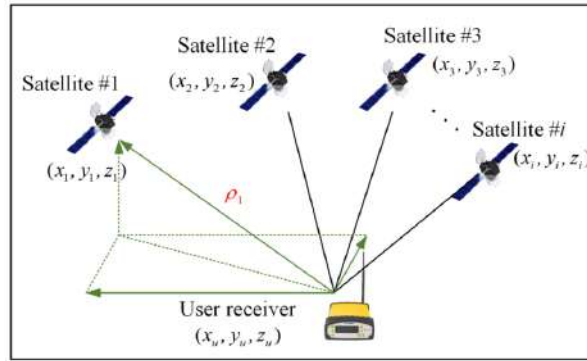


Figure 3: Schematic illustration of GNSS positioning, used under CC BY 4.0 license. [19]

2.2.1 Standard GPS

Standard GPS receivers typically achieve horizontal accuracies on the order of several meters (while standard in industrial applications, military systems are significantly more advanced). The level of accuracy is more than adequate and developed for waypoint navigation, but is clearly inefficient for precision landing.

2.2.2 Differential and RTK GPS

Seeing it from a theoretical standpoint, GNSS positioning is based on pseudo-range measurements; basically the satellite provides the receiver with the transmission time of a navigation message and the receiver estimates its distance to the satellite as

$$\rho_i = c(t_r - t_{s,i}) + b_r + \varepsilon_i, \quad (2.1)$$

[4] where c is the speed of light, meanwhile ρ_i is the "pseudo-range" to satellite i , t_r and $t_{s,i}$ are the reception and transmission times, b_r is the receiver clock bias and ε_i groups atmospheric, multipath and hardware errors. By combining measurements from at least four satellites, the receiver solves for its three-dimensional position and clock bias.

Standard GNSS receivers typically achieve horizontal accuracies of a few metres, while RTK techniques can reduce the error to the centimetre level under favourable conditions by exploiting carrier-phase measurements and differential corrections. However, the critical point of this approach is the need for a base station which processes all communications and positions, creating a significant problem in case of latency in the signal or cluttered environments. [4].

2.3 Ultra-Wideband Positioning Systems

Moving to relative positioning systems, the first one analyzed is the UWB ranging, which is typically obtained from the time-of-flight of short radio pulses; neglecting clock offsets, the distance between two nodes can be approximated as [5]

$$d \approx c \Delta t, \tag{2.2}$$

where d is the range, c is the speed of light and Δt is the measured signal propagation time. More advanced schemes (such as two-way ranging) explicitly compensate for clock drift and processing delays to achieve sub-decimeter accuracy.

By combining multiple fixed anchors with known positions, UWB systems can estimate the three-dimensional position of a mobile tag via trilateration, leading to a good resulting accuracy (on the order of 10–20 cm) indoors, with good robustness to multipath [5]. Nevertheless, this approach requires the installation and calibration of several fixed anchors around the landing area, which conflicts with the goal of a self-contained landing system that can operate on arbitrary moving platforms without additional infrastructure.

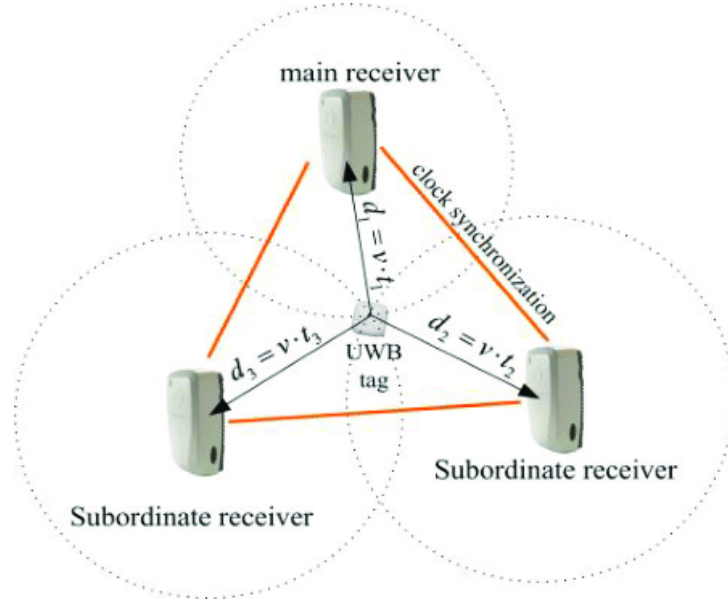


Figure 4: UWB positioning method, used under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License (<https://creativecommons.org/licenses/by-nc-nd/4.0/>). [20]

2.4 Inertial Measurement Units

Inertial Measurement Units (IMUs) are the core of a drone's sensor suite, providing high-rate measurements of linear acceleration and angular velocity, from their integration it is possible to estimate position and velocity over short time intervals. Their main limitation consists in their drift; as with any robot, inertial navigation suffers from unbounded drift due to sensor biases and noise, which increase exponentially over time [1]. For this reason the IMUs are primarily

used for state propagation and must be aided by external measurements to maintain long-term accuracy or, as in our case, embedded in the sensor architecture to boost our vision approach.

2.5 Vision-Based Positioning

This leads to vision-based positioning systems. It estimates relative pose using images captured by onboard cameras, which leads to many advantages as: high spatial resolution and particularly effective at short distances.

Vision-based methods can be broadly categorized into feature-based, model-based and fiducial marker-based approaches. However, in this thesis only the fiducial marker-based approach is considered. [2], [21].

2.5.1 Fiducial Markers for Pose Estimation

As we were saying, fiducial markers are artificial visual patterns designed to be easily detectable and uniquely identifiable. Using known geometric properties, it is possible to estimate the full six-degree-of-freedom pose of the marker relative to the camera. Over the years, several marker families have been proposed, each optimized for a different trade-off between robustness, payload capacity and computational cost [2] [22].

- **QR codes** were originally developed for industrial identification and consumer applications. They are capable of encoding large amounts of alphanumeric data thanks to error-correcting codes, but their detection pipeline is primarily optimized for reliable decoding rather than accurate corner localization. As a consequence, they are less suited to high-precision pose estimation and real-time control tasks (especially when the marker occupies only a small portion of the image).

- **AprilTags** are square binary markers explicitly designed for robotics and augmented reality. They employ a carefully constructed coding scheme with strong error detection and correction capabilities [22], which yields very low false-positive rates even under significant image noise. AprilTags are widely used in SLAM and calibration applications where robustness is prioritised over pure detection speed.
- **ArUco markers** follow a similar square-binary design of AprilTags, but are tightly integrated into the OpenCV library [2], [23] and provide a flexible set of predefined and custom dictionaries. The detection pipeline is comparatively lightweight, enabling real-time operation on embedded platforms while still offering reliable corner estimation. This makes ArUco markers particularly attractive for onboard UAV applications where computational resources are limited and very useful for our application.
- **Multi-scale and nested markers** extend the above families by embedding markers of different sizes into a single target. A large outer pattern guarantees detectability at high altitudes, while a smaller inner marker provides increased accuracy when the UAV is close to the platform. Such designs directly address the loss of resolution and field-of-view issues encountered by monocular cameras during the final landing phase [2], [24]. Several recent systems combine visual markers with additional modalities such as infrared beacons to increase robustness and tolerate single-sensor failures [21].

In the context of this thesis, a nested ArUco target is adopted. This choice leverages the efficient OpenCV implementation of ArUco markers while exploiting a multi-scale layout to

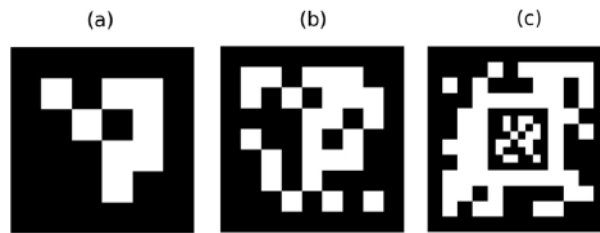


Figure 5: ArUco markers generated for the target frame using the OpenCV library in Python. [2]

maintain visibility at high altitude and achieve high local precision near touchdown, as detailed in Chapter 4.

2.6 ArUco Marker System

ArUco markers are square binary fiducial markers composed of an internal binary pattern, denoted as a black border. They are widely used in robotics and augmented reality due to their simplicity, efficiency and integration with the OpenCV library, Figure 5.

Once a marker is detected, its pose relative to the camera is estimated by analyzing a Perspective-n-Point (PnP) problem. Given the known marker size and the intrinsic parameters of the camera, the rotation matrix $\mathbf{R}$ and translation vector $\mathbf{t}$ are computed.

The estimated translation vector provides the relative position of the target in the camera frame, which can be transformed into the vehicle body frame for control purposes [2].

2.6.1 Marker Detection Pipeline

The ArUco detection pipeline follow these steps:

1. Image preprocessing and thresholding

2. Detection of square contours
3. Perspective normalization
4. Binary pattern decoding and marker identification

This pipeline allows for real-time detection even on computer systems with limited resources [2].

Table II summarizes some typical ArUco dictionaries used in the literature and highlights the trade-off [2] between code length and detection speed.

TABLE II: EXAMPLE OF ARUCO DICTIONARIES AND QUALITATIVE PROPERTIES [2].

Dictionary	Bits	# IDs	Detection effort
DICT_4X4_50	4×4	50	Low
DICT_5X5_100	5×5	100	Medium
DICT_6X6_250	6×6	250	Higher

In this work the DICT_4X4_50 dictionary is adopted [2] for both the outer and inner markers of the nested target. The short code length reduces the number of pixels required for a reliable detection and keeps the decoding computationally light, which is crucial to sustain frame rates around 30 Hz on embedded hardware while still providing enough unique IDs for the considered scenarios.

2.6.2 Accuracy and Limitations

ArUco-based pose estimation offers high accuracy at short distances, however the accuracy degrades with increasing distance due to reduced pixel resolution and image noise. Additionally, monocular vision systems are sensitive to:

- Motion blur
- Lighting variations
- Partial occlusions

These limitations are particularly critical during the final landing phase.

2.6.3 Multi-Scale Marker Strategy

To mitigate field-of-view and resolution issues, a multi-scale marker strategy is adopted. A larger marker ensures detectability at higher altitudes, while a smaller marker provides increased precision during the final descent.

This approach effectively addresses the trade-off between detection range and positioning accuracy [21].

2.7 Summary

The overall analysis indicates that a vision-based relative positioning system based on ArUco markers offers the best compromise between accuracy, robustness and deployment cost for the considered application. Unlike GNSS or UWB solutions, the proposed approach does not require external infrastructure and scales naturally to different landing sites. At the same time, the use

of a nested ArUco target with a multi-scale detection strategy enables high local precision during the final approach, directly addressing the limitations identified for standard monocular vision at low altitude.

TABLE III: COMPARISON OF POSITIONING TECHNIQUES FOR UAV LANDING

Method	Accuracy	Infrastructure	Suitability
Standard GPS	Low	None	Navigation
RTK GPS	High	Required	Limited
UWB	High	Required	Indoor
ArUco Vision	High (local)	None	Precision landing

CHAPTER 3

PX4 AUTOPILOT AND SYSTEM ARCHITECTURE

3.1 System Overview

The core of the flight control system is based on **PX4 Autopilot**, which is an open-source flight control software mostly used in drones and other unmanned vehicles. PX4 gives to the user a modular architecture that abstracts the underlying hardware [9], allowing the same codebase to run on various flight controllers (Pixhawk 6C) and in simulated environments like Gazebo Harmonic.

3.1.1 Flight Stack and Middleware

The PX4 architecture is built upon a middleware layer called **uORB** (Micro Object Request Broker); it acts as an asynchronous publish-subscribe messaging bus, allowing different modules (sensors, estimators, controllers) to exchange data without tight coupling [9].

This modular architecture is particularly advantageous because it decouples state estimation, low-level stabilization and high-level navigation from PX4 allows the proposed vision-based landing controller to be integrated as an additional module that operates on top of the existing flight stack. Basically the same visual landing logic can be executed both in the Gazebo Harmonic simulation and on the real UAV without modifying the inner attitude and rate controllers, which significantly simplifies deployment and reduces the risk associated with custom low-level control software.

3.1.2 Control Stack

PX4 implements a cascaded control architecture in which each layer regulates a different subset of the vehicle dynamics [9]. The outer position controller computes a velocity setpoint (v_x, v_y, v_z) from the position error, then the velocity controller translates this command into attitude and thrust setpoints (ϕ, θ, T) ; at the end the inner rate controller drives the motor outputs to track the desired angular rates. [9] This structure allows high bandwidth for attitude stabilization while keeping the position loop slower and more robust to sensor noise.

During the landing phase, the proposed approach overrides only the outermost position loop in the horizontal plane. The custom vision-based controller directly generates body-frame velocity commands in the x - y plane, which are injected into the PX4 velocity controller via Offboard mode. The nominal altitude controller and the inner velocity and rate loops remain unchanged, so that low-level stability and actuator handling are still managed by the standard PX4 stack [9]. This particular design choice limits the scope of the modifications while preserving the well-tested inner control architecture and allowing the visual landing algorithm to focus solely on image-plane tracking and horizontal guidance.

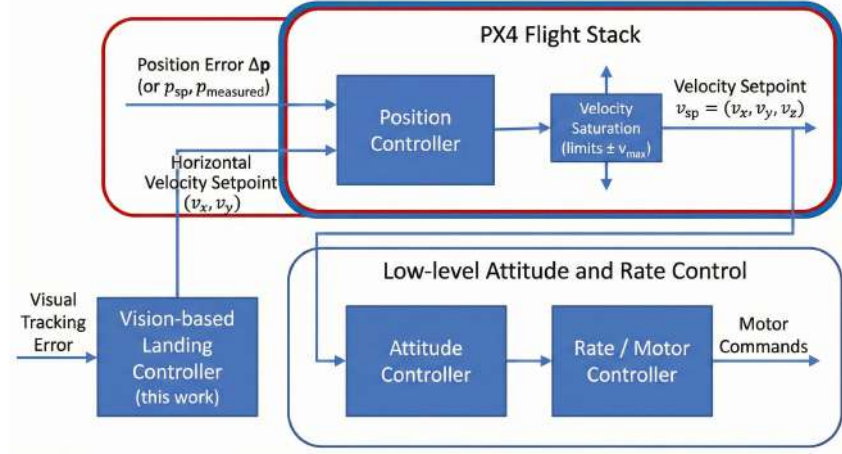


Figure 6: Schematic view of the PX4 cascaded control architecture and the insertion point of the proposed visual landing controller.

3.2 Offboard Control Interface (MAVSDK)

While other works rely on a full ROS2 stack to implement modular landing architectures [6], in this thesis a lighter MAVSDK-based design is adopted to minimise latency and deployment complexity; the system leverages **MAVSDK** (MAVLink SDK) in a Python environment [25].

The control logic runs on an external node (Companion Computer or Simulator) and communicates with the PX4 Flight Stack via the **MAVLink** protocol over a UDP (simulation) or UART (hardware) connection.

The architecture is designed as follows:

- **Vision Node:** Captures images and estimates the target position relative to the drone frame.
- **Control Logic:** A custom State Machine (implemented in Python) processes the visual error using a Linear Kalman Filter and a PID controller.
- **Command Injection:** The computed velocity commands $(v_x, v_y, v_z, \dot{\psi})$ are sent to the autopilot at 100Hz using the `Offboard::set_velocity_body()` method [25].

In this work, MAVSDK (MAVLink SDK) is adopted instead of a full ROS 2 middleware. This because it provides a lightweight, single-process API with direct access to MAVLink messages, which reduces communication jitter and processing overhead compared to a multi-node ROS 2 deployment [25]. The offboard landing controller executes as an asynchronous Python process using MAVSDK. The main control loop runs at $F_{\text{ctrl}} = 100$ Hz, while the visual measurement thread operates at approximately 30 Hz. Both threads share a buffer that stores the most recent marker detection and image frame, implementing a zero-order hold of the visual information between updates. At each control cycle, the controller reads the latest available measurement, updates the Kalman filter state, and sends a body-frame velocity command to PX4 via `set_velocity_body(VelocityBodyYawspeed)`. The resulting architecture, summarised in Figure 6, allows the proposed vision-based landing controller to override the nominal horizontal position loop while preserving the standard PX4 low-level attitude and rate controllers described in Chapter 5.

3.3 World and Body Frame Modifications

The simulation environment is implemented in Gazebo Harmonic (version 8.10), which provides an updated physics engine and rendering pipeline compared to the classic Gazebo distribution [26], [27].

3.3.1 Coordinate Frames

Two main coordinate systems are utilized:

- **NED (North-East-Down):** The local world frame used by PX4 navigation. X points North, Y East, and Z Down (gravity direction).
- **FRD (Front-Right-Down):** The body-fixed frame attached to the drone. X points forward (camera direction), Y right, and Z down.

Visual measurements are first obtained in the camera frame, which is rigidly attached to the UAV. The camera frame differs from the FRD body frame by a fixed rotation R_{cb} and translation p_{cb} , determined by the physical mounting of the camera. The detected marker position is therefore transformed from camera to body frame using

$$p^b = R_{cb} p^c + p_{cb} \quad (3.1)$$

and then expressed in the local NED frame through the standard PX4 attitude rotation. In this way, the image-plane error can be consistently converted into body-frame velocity commands that are compatible with the PX4 velocity controller.

3.3.2 Simulation Environment Customization

To replicate the experimental conditions, the default assets were modified:

- **Drone Model (`gz_x500_vision`):** The UAV model is based on the `gz_x500_vision` template, which was extended by adding a downward-facing camera matching the characteristics and mounting position of the real hardware. In particular, the virtual camera is mounted with the same geometric offset with respect to the vehicle centre of mass, so that the parallax correction and control gains tuned in simulation can be directly reused on the physical platform.



Figure 7: Simulated UAV model (`gz_x500_vision`) on the nested ArUco landing pad in Gazebo Harmonic.

- **World File (default.sdf):** The world file is derived from the default Gazebo `.sdf` scene, enriched with a high-contrast red ground plane to improve visual segmentation of the landing target. Lighting parameters are adjusted between simulation runs to emulate different outdoor conditions and to verify the robustness of the ArUco detection pipeline under varying illumination. Specific Gazebo Harmonic system plugins (`Physics`, `Sensors`, `SceneBroadcaster`) were manually injected to ensure correct synchronization between the physics engine (running at 1000Hz) and the vision loop (running at 30Hz) [24].
- **Moving Platform (moving_platform.sdf):** The moving platform model is obtained by modifying a standard mobile base to resemble the TurtleBot used in the real experiments, both in footprint and visual appearance. It is modelled as a square structure composed of three nested regions: the inner ArUco target, a neutral grey buffer and an outer orange frame. The high-saturation orange background is chosen to provide a strong colour contrast with both the black marker and the green ground plane, reducing the risk of visual washout and improving the robustness of the thresholding stage in the ArUco detection pipeline. The UAV always takes off from the platform itself, so that the entire mission (from departure to touchdown) is executed within the same visual context as in the real experiments [26].

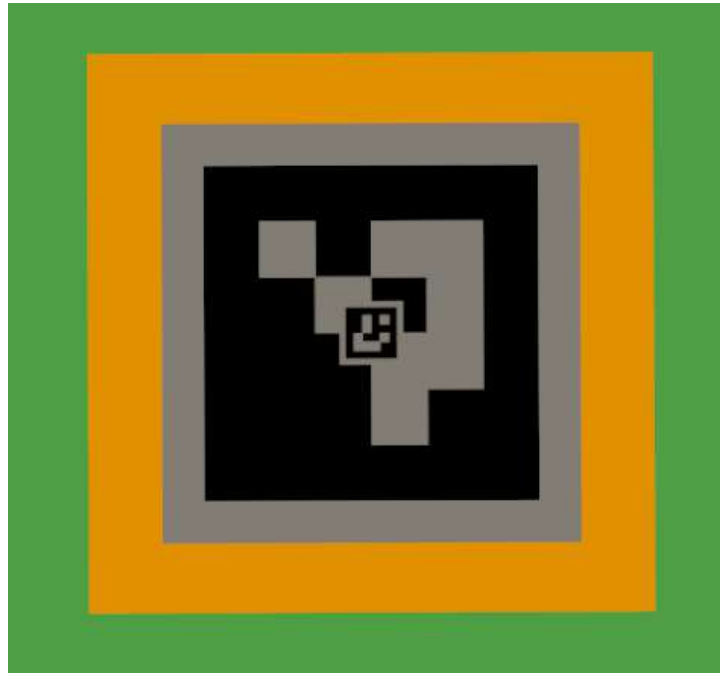


Figure 8: Top view of the coloured landing pad with nested ArUco markers used in simulation.

The physics engine in Gazebo Harmonic is configured to run at 1000 Hz, while the vision loop operates at 30 Hz with an image resolution of 640×480 pixels. Specific Gazebo Harmonic system plugins (**Physics**, **Sensors**, **SceneBroadcaster**) are enabled to ensure consistent time stamping between simulated sensor data and PX4 SITL, so that the latency and sampling characteristics observed in simulation closely match those of the real system. The moving platform, implemented in the `moving_platform.sdf` model, follows predefined trajectories with

speeds comparable to those used in the experimental trials described in Chapter 7, enabling a meaningful sim-to-real comparison.

CHAPTER 4

FILTERING AND SENSOR FUSION

4.1 Software Adaptation

While the core logic (State Machine, Kalman Filter, PID Controller) remains identical, the Input/Output interfaces must be adapted:

1. **MAVSDK Connection:** The connection string in the Python script changes from the simulated UDP port (`udp://:14540`) to the physical serial device (`serial:///dev/ttyTHS1:1000000`).
2. **Video Source Abstraction:** In the simulation, images are retrieved via *Gazebo Transport* topics. In the real application, the `vision_callback` function is modified to capture frames directly from the physical camera device (e.g., `/dev/video0`) using OpenCV:

```
# Simulation:

node.subscribe(Image, "/camera", vision_callback)


# Real World:

cap = cv2.VideoCapture(0)

ret, frame = cap.read()
```

3. **Shared Buffer and Thread Safety:** A further structural difference between the two implementations concerns the thread safety of the shared buffer. In the simulation script, the vision callback is invoked directly by the Gazebo Transport subscription mechanism within the same process context; since the callback and the main control loop execute sequentially within the asynchronous event loop, no explicit synchronisation is required. In the real-world deployment, however, the camera acquisition runs on a dedicated Python thread (`CameraThread`) that continuously captures frames from the USB device. Because the producer (camera thread) and the consumer (asynchronous control loop) operate concurrently, the `SharedBuffer` class in `REAL_landing.py` protects every read and write operation with a threading, lock to prevent data races. This modification is confined to the I/O layer and does not alter the control logic: in both cases the buffer semantics remain those of a zero-order hold, exposing the most recent valid measurement to the controller at each cycle.

In both cases, the same landing controller is executed with minimal changes to the Input/Output layer. The core logic (state machine, Kalman filter, PID and feed-forward terms) remains identical across simulation and real flights, which greatly simplifies validation and reduces the risk of software regressions during the sim-to-real transfer [9]. Only the input/output layers are adapted between simulation and real flights. In particular, the transport mechanisms (UDP vs. UART) and the image acquisition back-ends (Gazebo virtual camera vs. physical USB camera) are changed, while the state machine logic, Kalman filter structure and tuning, as well as the PID gains, are kept identical. This strategy facilitates a direct sim-to-real comparison and

ensures that the observed performance differences can be primarily attributed to sensor noise and environmental factors rather than to changes in the control law. The parallax correction described in Equation 5.5 is applied identically in both the simulation and the real-world scripts: the expected pixel offset is added to the longitudinal image-plane error e_y at each control cycle, compensating for the physical displacement between the camera and the vehicle centre of mass.

4.2 Pinhole Camera Model and Pose Estimation

The relationship between 3D world coordinates and their projection onto the image plane is described by the pinhole camera model. Let a point $P_c = [X_c, Y_c, Z_c]^\top$ be expressed in the camera frame. Its projection onto pixel coordinates (u, v) is:

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \frac{1}{Z_c} K P_c, \quad \mathbf{K} = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \quad (4.1)$$

where f_x, f_y represent the focal lengths in pixels along each axis, meanwhile (c_x, c_y) is the principal point (ideally the image centre). For the camera used in this work (640×480 resolution), the calibrated parameters are $f_x \approx f_y \approx 550$ px and $(c_x, c_y) \approx (320, 240)$ px.

4.2.1 Pixel-to-metre ratio.

At altitude h , a physical displacement Δd on the ground plane maps to a pixel displacement:

$$\Delta u = \frac{f \cdot \Delta d}{h} \quad (4.2)$$

This relationship is central to understanding the optical gain explosion phenomenon: as $h \rightarrow 0$ the pixel-to-metre ratio f/h diverges, meaning that even small physical displacements produce large pixel errors. The gain-scheduling strategy described in Chapter 5 directly counteracts this effect by reducing the PID gains as altitude decreases.

4.3 Visual Navigation Target Design

The autonomous landing system relies on visual feedback from ArUco markers, which are synthetic squared binary fiducial markers. To overcome the dynamic range limitations of a single marker, a custom *Nested Target* was designed and generated using a dedicated Python script.

4.3.1 Nested Target Architecture

The nested target is composed of two hierarchical identifiers designed to handle different flight phases:

- **Primary Marker (ID 0):** A large marker with a side length of 1600 pixels (in the generated image). This marker is intended for the initial acquisition phase at altitudes between 10m and 2m.
- **Secondary Marker (ID 4):** A smaller marker with a side length of 250 pixels, placed at the exact geometric center of the primary one. It provides millimetric precision during the final approach (below 1.5m).

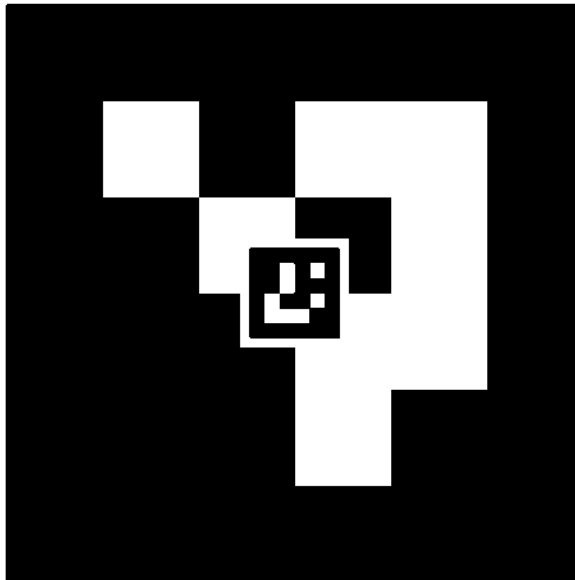


Figure 9: Nested target used in our research generated for the target frame using the OpenCV library in Python.

The nested landing target consists of a large outer ArUco marker and a smaller inner marker, printed on a single planar board. Table IV summarises the main physical dimensions adopted in this work.

TABLE IV: PHYSICAL DIMENSIONS OF THE NESTED ARUCO LANDING TARGET.

Parameter	Value	Description
Outer marker side length	$L_{\text{out}} = 0.50 \text{ m}$	Detection at high altitude
Inner marker side length	$L_{\text{in}} = 0.15 \text{ m}$	High precision near touchdown
Board side length	0.60 m	Coloured background area
Border ratio	1.2	Black border width relative to code

At an altitude of approximately 1.5 m and with the onboard camera resolution of 640×480 pixels, the inner marker still occupies several tens of pixels per side, which is sufficient for reliable corner localisation. The selected border ratio of 1.2 was validated experimentally under different illumination conditions and moderate motion blur to prevent merging between the inner marker bits and the outer pattern during adaptive thresholding, while preserving a sufficiently large active area for detection at short range.

4.3.2 Vision Debugging Tool

To support development and tuning, a lightweight vision debugging tool was implemented. This tool overlays the detected marker contours, IDs and the image-plane error (e_x, e_y) on the camera stream, and logs the detection confidence and processing time for each frame. It is executed in parallel to the main controller and is used only during indoor tests and simulation runs, without affecting the real-time execution of the landing algorithm [23].

4.4 State Estimation via Kalman Filter

To mitigate the noise introduced by visual detection, a linear Kalman Filter was implemented [28]. The system state at step k is defined by the discrete-time vector:

$$x_k = \begin{bmatrix} x \\ \dot{x} \\ y \\ \dot{y} \end{bmatrix}_k \quad (4.3)$$

where x and y denote the image-plane coordinates of the marker center in pixels, and $\dot{x}$, $\dot{y}$ are the corresponding pixel velocities. Under a constant-velocity assumption with a sampling time $\Delta t = 1/100$ s, the prediction step is formulated as:

$$x_{k|k-1} = F x_{k-1|k-1} \quad (4.4)$$

with the state transition matrix F correctly discretized as [28]:

$$F = \begin{bmatrix} 1 & \Delta t & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & \Delta t \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (4.5)$$

The process and measurement noise covariances are chosen as

$$Q = 0.02 * I_4, \quad R = 15.0 * I_2, \quad (4.6)$$

prioritising smooth velocity estimates over raw pixel measurements, in accordance with the tuning procedure. A detailed discussion of the selected matrices Q and R and their impact on the estimator performance is provided in Chapter 6, where the simulation results are analyzed. The Kalman filter implements a constant-velocity model in the image plane, which is particularly suited to compensate for sensing and communication delays. Assuming a vision-to-command delay of roughly two to three frames at 30 Hz, the prediction step propagates the estimated target position forward in time before each control update, thereby partially compensating for latency [28].

4.4.1 Asynchronous Communication and Zero-Order Hold

In the implementation, visual measurements are produced by a dedicated callback that decodes the Gazebo image stream, detects ArUco markers and writes the marker centre into a shared buffer together with a time stamp. The control loop reads this buffer at 100 Hz; if a new measurement is available, the filter performs a correction step, otherwise it relies on prediction only. This mechanism is equivalent to a zero-order hold on the last valid measurement and decouples the vision frame rate from the control frequency without blocking the controller when no detection is available.

A time-out on the last detection time is used to declare the target as lost; if no valid measurement is received for more than 1.5 s the target is declared not visible and the controller freezes the current descent rate and horizontal commands (anti-glitch phase). If the target remains undetected for a total of 3 s, the state machine transitions to the active search behaviour described in Section 5.

CHAPTER 5

SYSTEM CONTROL ALGORITHM

5.1 System State Machine

The overall landing behaviour is organized as a finite state machine and the main transitions and actions are summarized in Figure 10. The main states are: *Idle*, *Target Acquisition*, *Tracking*, *Final Descent*, *Touchdown*, and *Search Mode*. During *Idle* the UAV maintains a safe hover while waiting for the landing command. In *Target Acquisition* the controller searches for the outer ArUco marker and initializes the Kalman filter once a valid detection is available.

When the marker is consistently detected, the system transitions to the *Tracking* state, where the PID controller with feedforward compensates the relative motion of the platform. As the altitude decreases and the inner marker becomes dominant, the controller enters the *Final Descent* state, using the dynamic alignment threshold and gain scheduling described in Chapter 4 to ensure a smooth approach. If the touchdown conditions on altitude and alignment are met, the *Touchdown* state is triggered and the kill command is issued.

If at any point the fiducial marker is lost for longer than a predefined timeout, the state machine switches to *Search Mode*. In this state the UAV stops descending and executes the spiral search and climbing behaviour detailed in Subsection 6.5, until the marker is re-acquired or a safety timeout is reached.

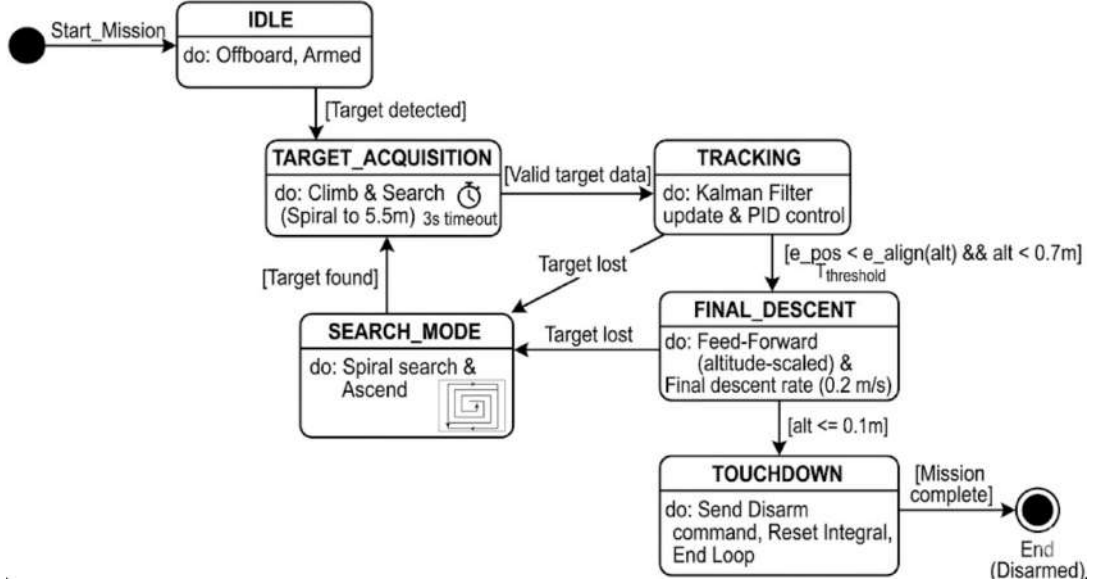


Figure 10: State machine of the landing algorithm.

5.2 PID Control Logic and Geometric Correction

The velocity commands sent to the drone via MAVSDK map the image-plane errors to the proper body-frame axes. Due to the camera frame orientation, the vertical pixel error e_y drives the pitch velocity v_x , while the horizontal pixel error e_x drives the roll velocity v_y :

$$v_x(t) = - \left(K_{p,y} \gamma(h) e_y(t) + K_{d,y} \gamma(h) \dot{e}_y(t) + K_i \int e_y(\tau) d\tau + K_{ff}(h) \hat{v}_y(t) \right) \quad (5.1)$$

$$v_y(t) = K_{p,x} \gamma(h) e_x(t) + K_{d,x} \gamma(h) \dot{e}_x(t) + K_i \int e_x(\tau) d\tau + K_{ff}(h) \hat{v}_x(t) \quad (5.2)$$

where e_x, e_y denote the image-plane position error and $\hat{v}_x, \hat{v}_y$ the corresponding velocity estimates provided by the Kalman filter. The commanded horizontal velocities are additionally clamped by an altitude-dependent speed limit to prevent excessively aggressive manoeuvres near the ground:

$$v_{\max}^{xy}(h) = \text{clip}(0.8 h, 0.1, 1.4) \text{ m/s}. \quad (5.3)$$

This ensures that the maximum horizontal speed is reduced to a minimum of 0.1 m/s at low altitudes and progressively increases to 1.4 m/s at higher altitudes. To avoid excessive integral action close to touchdown, the integral term is enabled only above a safety altitude $h_{\text{cut}} = 0.8 \text{ m}$ and is subject to standard anti-windup saturation [27]:

$$\left| \int e_{x,y}(\tau) d\tau \right| \leq I_{\max} \quad (5.4)$$

This design prevents large corrective commands in the last meter of descent while still removing steady-state errors at higher altitudes. In order to compensate for the parallax effect, introduced by the physical offset of the camera relative to the vehicle's center of mass, the measured pixel error is corrected. Assuming an offset O_y along the vehicle's forward axis and O_z along the vertical axis, the correction is applied as:

$$offset_{pix} = \frac{O_y f}{\max(h - O_z, h_{\min})} \quad (5.5)$$

$$e_{y,corr} = e_y + offset_{pix} \quad (5.6)$$

where f is the focal length in pixels and $h_{\min}$ is a lower bound threshold used to prevent numerical singularities right at touchdown. In order to dynamically adjust the controller's aggressiveness based on the altitude, a damping factor $\gamma(h)$ is introduced. This term linearly scales the proportional and derivative gains during the final approach, saturating between 40% and 100%:

$$\gamma(h) = \max \left(0.40, \min \left(1.0, \frac{h - 0.5}{1.2} \right) \right) \quad (5.7)$$

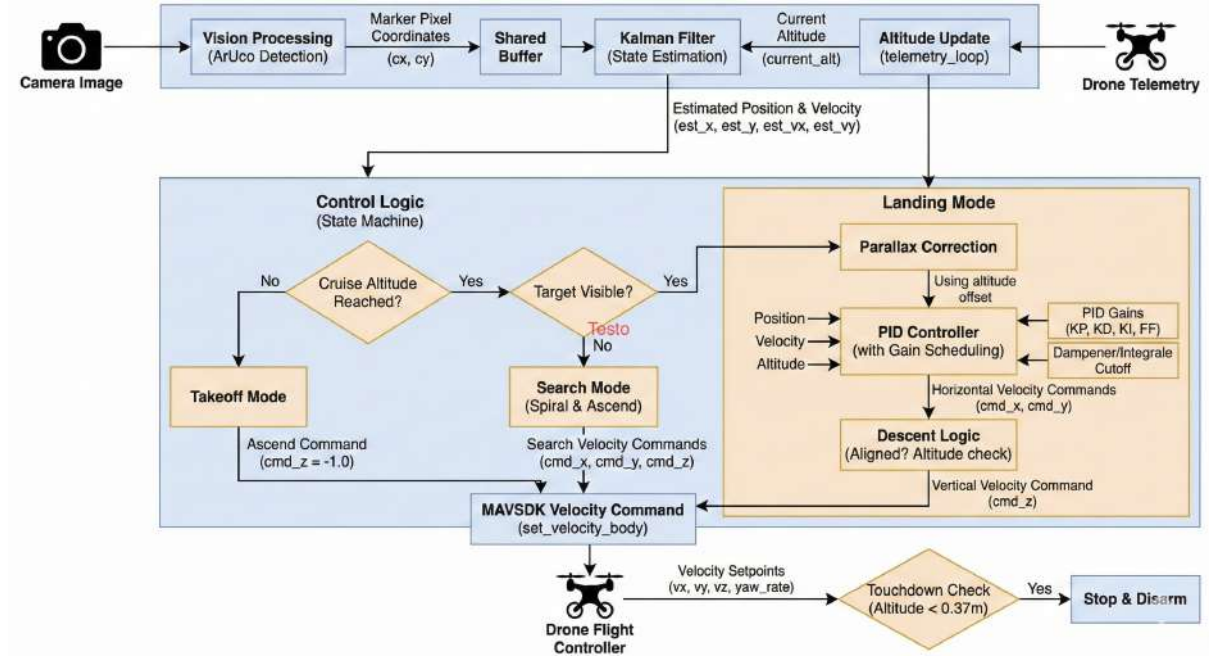


Figure 11: High-level architecture of the landing controller.

5.2.1 Integral action and anti-windup

The integral term is handled to remove residual steady-state offsets caused by bias in the visual measurements or small modelling errors. To avoid excessive overshoot near touchdown, the accumulation rate of the integral action is linearly damped from 100% to 30% of its nominal value as the altitude decreases from 3.0 *m* to 1.0 *m*. Below $h = 0.4$ *m* the integral is completely disabled ($i_{\text{damp}} = 0$) to prevent any residual wind-up from destabilising the final approach. The integral state is also bounded by symmetric limits, implementing a standard anti-windup scheme.

5.2.2 Feed-Forward Factor

The feed-forward gain is modulated by the magnitude of the image-plane tracking error rather than by altitude. When the UAV is far from the target centre in the image plane, the full gain $K_{FF}^{\max}$ is applied; as the error decreases below 135 px the gain is linearly reduced, and it is completely disabled below 50 px to prevent overshoot during the final centering phase. Rather than scheduling the feed-forward gain with altitude alone, the implementation modulates it with the current image-plane error magnitude $d_e = \sqrt{e_x^2 + e_y^2}$ (in pixels). A spatial multiplier

$$\alpha(d_e) = \text{clip}\left(\frac{d_e - 20}{85}, 0, 1\right) \quad (5.8)$$

scales the maximum feed-forward gain $K_{FF}^{\max} = 0.0035$:

$$K_{FF}(d_e) = K_{FF}^{\max} \alpha(d_e). \quad (5.9)$$

When the marker is well centred ($d_e < 20$ px) the feed-forward is deactivated, preventing overshoot near the target centre. As the tracking error grows, α ramps linearly to unity at $d_e = 105$ px, providing full anticipatory compensation for the platform motion.

This heuristic proved effective in reducing steady-state error at higher altitudes while preserving a smooth final descent. A summary of the main controller and estimator parameters used in the simulations is reported in Table V.

5.2.3 Gain Scheduling and Dynamic Alignment Threshold

Both the proportional derivative gains and the maximum allowable horizontal speed are scaled with the current altitude via the damping factor $\gamma(h)$ defined in (Equation 5.7), which reduces the controller aggressiveness near the ground.

The effective alignment threshold is obtained by multiplying ε_0 by an altitude-dependent cone factor $\kappa(h)$, implemented as a piecewise-linear interpolation:

$$\kappa(h) = \text{interp}(h, [0.7, 2.0, h_{\text{target}}], [1.0, 1.5, 2]), \quad (5.10)$$

so that

$$\varepsilon_{\text{align}}(h) = \kappa(h) \varepsilon_0. \quad (5.11)$$

At $h \leq 0.7$ m the threshold equals the nominal value $\varepsilon_0 = 110$ px; at $h = 2.0$ m it is relaxed to $1.5 \varepsilon_0 = 165$ px; and at cruise altitude ($h \geq h_{\text{target}} = 5.5$ m) it reaches $2 \varepsilon_0 = 220$ px. At cruise altitude ($h \geq 2.0$ m) each pixel subtends a relatively large physical area; consequently, an error of $2 \varepsilon_0 = 220$ px still corresponds to an acceptable real-world offset and the UAV can initiate descent without performing an exhaustive centering manoeuvre. As altitude decreases, the pixel-to-metre ratio increases and each pixel becomes a more sensitive indicator of physical misalignment. The threshold therefore narrows progressively, reaching $\varepsilon_0 = 110$ px at $h \leq 0.7$ m, where precise centering is required to guarantee a safe touchdown within the boundary of the landing pad.

Descent is commanded only when both tracking error components satisfy the threshold simultaneously:

$$|\hat{e}_x| < \varepsilon_{\text{align}}(h) \quad \text{and} \quad |\hat{e}_y| < \varepsilon_{\text{align}}(h), \quad (5.12)$$

where $\hat{e}_x, \hat{e}_y$ are the Kalman-filtered image-plane errors. If either condition is violated the vertical velocity command is set to zero and the UAV hovers at the current altitude while the PID controller continues to reduce the lateral error. The final landing trigger is independent of the alignment check and is issued when the relative altitude falls below $h_{\text{touchdown}} = 0.20 \text{ m}$ (see Section 5.3).

This two-stage design-relaxed threshold for early descent, strict threshold near the ground-proved essential to prevent limit-cycle behaviour: at high altitude, forcing tight pixel alignment would delay the descent unnecessarily; at low altitude, maintaining a loose tolerance would risk a lateral contact outside the safe landing zone.

5.3 Landing Phase

The final landing phase represents the critical transition between dynamic tracking and physical contact with the platform. In this phase, the primary objective of the controller is to guarantee a soft and immediate touchdown, minimizing the vehicle's exposure to ground effect turbulence and the loss of resolution in the vision system.

The transition to the landing sequence is governed by a dual-threshold logic that integrates telemetric and visual data:

1. **Altitude Condition:** The vehicle must be below a predefined safety altitude $h_{touchdown}$.

In the SITL simulation software, this value is set to 0.20 m to compensate for the precision of the onboard sensors and to prevent bouncing on the platform.

2. **Alignment Condition:** The center of the nested ArUco marker must be within the dynamic alignment threshold ϵ_{align} . As implemented in the `REAL_landing.py` module, the drone is considered ready for impact only if the planar error estimated by the Kalman filter is less than 110 pixels (scaled according to the current altitude).

5.3.1 Descent Logic and Land Command

During the final approach ($1.5 \text{ m} > h > h_{touchdown}$), the vertical descent velocity v_z is not constant, but is modulated through a linear interpolation to ensure an asymptotic approach:

$$v_z = \text{interp}(h, [0.35, 1.5], [0.45, 0.8]) \text{ m/s} \quad (5.13)$$

This logic, embedded in the `run()` function of both software scripts, allows the drone to decelerate from 0.8 m/s at 1.5 m to 0.45 m/s at 0.35 m, thereby easing the task of the PID controller in maintaining centralisation during the final approach.

5.3.2 Termination Sequence (Touchdown)

Once both the altitude and alignment conditions are simultaneously met, the system executes a rapid disarming sequence to cut off motor thrust:

- **Command Zeroing:** The controller sends a final null horizontal velocity setpoint and a forced descent velocity (0.5 m/s) via the `set_velocity_body()` method to ensure firm physical contact.
- **Landing Trigger:** The `drone.action.land()` command is invoked (or, in more aggressive configurations, the `kill()` command via MAVSDK). This action explicitly interrupts the Offboard control loop and passes control to the internal PX4 land detector, which proceeds with the automatic disarming of the motors (locking) once the impact is detected.

This “defensive” architecture ensures that the drone does not attempt violent corrective maneuvers just a few centimeters above the platform, where optical glare or aerodynamic disturbances could otherwise cause the vehicle to tip over.

CHAPTER 6

SIMULATION

6.1 Estimator Tuning and Covariance Matrices

The Kalman Filter used for state estimation was tuned based on the empirical noise characteristics of the simulation environment [28]. As standard notation, in this work noise covariance matrix $\mathbf{Q}$ and measurement noise covariance matrix $\mathbf{R}$ were defined as:

$$\mathbf{Q} = \mathbf{I}_4 \cdot 0.02, \quad \mathbf{R} = \mathbf{I}_2 \cdot 15.0 \quad (6.1)$$

These values prioritize the stability of the velocity estimates $(\dot{x}, \dot{y})$ over the raw pixel measurements, effectively filtering out high-frequency noise from the ArUco detection during high-speed maneuvers.

6.1.1 Multi-Scale Detection Logic

The landing controller implements a priority-based detection logic to manage the transition between the nested markers. The algorithm constantly monitors the visibility of both IDs:

$$ID_{selected} = \begin{cases} 4 & \text{if } ID_4 \text{ is detected} \\ 0 & \text{if } ID_4 \text{ is not detected and } ID_0 \text{ is visible} \\ \text{None} & \text{if neither is detected (Trigger Search Mode)} \end{cases} \quad (6.2)$$

TABLE V: MAIN PARAMETERS OF THE VISION BASED LANDING CONTROLLER.

Parameter	Symbol / Value	Description
Kalman process noise	$Q = I_4 \cdot 0.02$	State propagation noise for $(x, \dot{x}, y, \dot{y})$
Kalman measurement noise	$R = I_2 \cdot 15.0$	Pixel measurement noise for (x, y)
Integral gain	$K_i = 0.0005$	Shared across both axes
Max horizontal speed	$v_{\max}^{xy} = \text{clipped}$	Altitude-dependent velocity clamp
Alignment threshold (nom.)	ϵ_0 (pixels)	Image-plane alignment threshold
Alignment threshold (low h)	$1.25 \epsilon_0$	Relaxed threshold for $h \leq 1.2$ m
Integral dampener range	1.2m to 0.2m,	Scales int accumulation down to 30%
Feed-forward gain (max)	$K_{FF}^{\max} = 0.0035$	Full gain when $d_e \geq 105$ px
Feed-forward ramp-on	$d_e \in [50, 135]$ px	Linear ramp from 0 to $K_{FF}^{\max}$
Target-lost declaration	1.5 s	Time without detection
Search mode activation	3.0 s	Total loss time before active spiral search
Search altitude	5.0 m	Altitude during spiral search phase
Touchdown alt	0.20 m	Alt threshold for kill condition
Threshold alignment	110 pixels	Image-plane alignment

During the landing phase, the controller evaluates a dynamic alignment threshold in the image plane, using Equation 5.11 where ϵ_0 denotes the nominal pixel threshold. The UAV is considered aligned if $|e_x| < \epsilon_{\text{align}}$ and $|e_y| < \epsilon_{\text{align}}$. Relaxing the threshold at low altitudes mitigates oscillations due to the increased pixel density close to the target. By prioritizing the smaller marker (ID 4), the system automatically increases the resolution of the position error as the UAV approaches the platform. Since both markers share the same geometric center, the handover between scales occurs without discontinuities in the estimated state, ensuring a smooth trajectory during the final descent. If the fiducial marker remains undetected for more than 3 s,

the controller switches from hovering to a spiral search mode while simultaneously commanding a slow ascent to a predefined search ceiling:

$$h_{\text{cmd}} = \begin{cases} \text{ascend at } 1.0 \text{ m/s,} & h < 5.0 \text{ m} \\ \text{hold,} & h \geq 5.0 \text{ m} \end{cases} \quad (6.3)$$

The ascent to 5.0 m enlarges the field of view and prevents the UAV from remaining trapped in a blind spot near the platform during the search phase.

6.2 Kalman Filter Sensitivity Analysis

The choice of the process noise covariance $Q = 0.02 \cdot I_4$ and measurement noise covariance $R = 15.0 \cdot I_2$ was made through systematic experimentation. To quantify the sensitivity of the landing accuracy to these parameters, a parametric study was conducted in simulation by varying Q and R independently while keeping all other controller parameters fixed.

6.2.1 Effect of Q

The process noise covariance governs how quickly the filter adapts to changes in marker velocity. Three values were tested: $Q \in \{0.005, 0.02, 0.10\}$, each multiplied by I_4 .

TABLE VI: SENSITIVITY OF LANDING ACCURACY TO THE PROCESS NOISE PARAMETER Q .

Q	RMSE_{total} [cm]	CEP [cm]	Velocity noise $\sigma_{\dot{x}}$ [px/s]	Velocity noise $\sigma_{\dot{y}}$ [px/s]
0.005	2.64	2.21	0.0388	0.0459
0.02	2.78	2.5	0.315	0.0471
0.10	4.65	4.28	0.1223	0.1043

All tests performed with $R = 15.0$, platform speed 1km/h, $N = 5$ trials each. With a low process noise ($Q = 0.005$), the filter is too sluggish and cannot track sudden target movements during the final approach, resulting in higher landing error. Conversely, a high process noise ($Q = 0.10$) makes the velocity estimate noisy, causing oscillatory control commands. The selected value $Q = 0.02$ offers the best compromise.

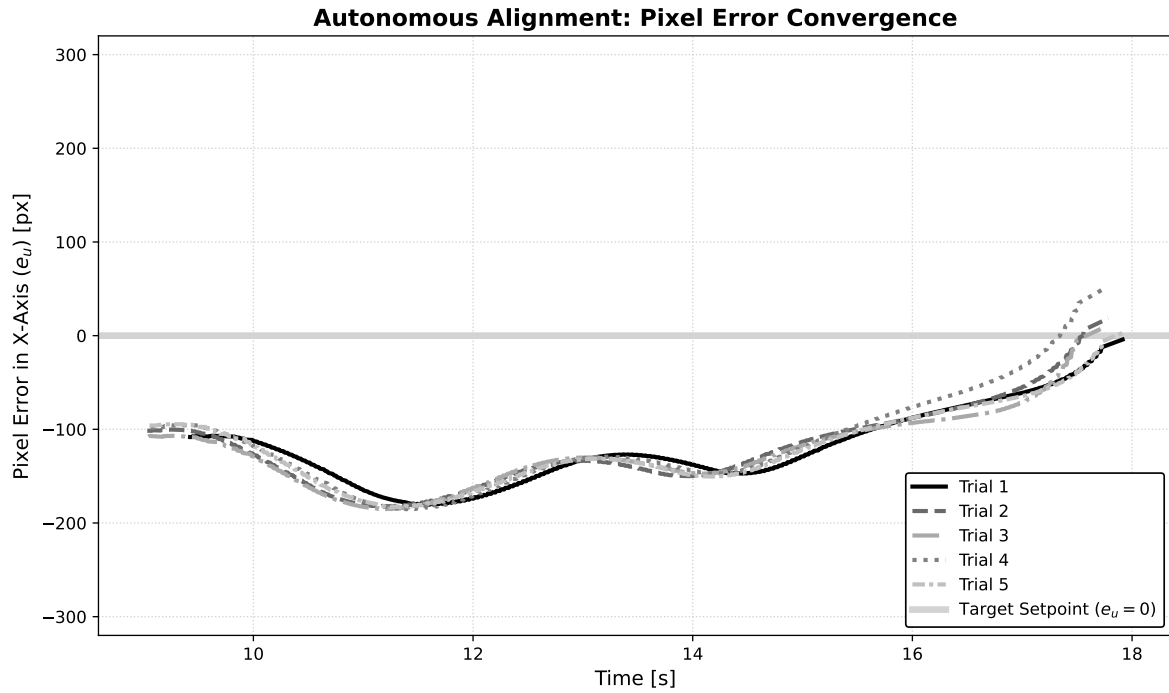


Figure 12: Effect of process noise covariance on the Kalman filter.

6.2.2 Effect of R

The measurement noise covariance determines how much the filter trusts individual ArUco detections. Three values were tested: $R \in \{5.0, 15.0, 50.0\}$.

TABLE VII: SENSITIVITY OF LANDING ACCURACY TO THE MEASUREMENT NOISE
PARAMETER R .

R	$\text{RMSE}_{\text{total}}$ [cm]	CEP [cm]
5.0	5.5	4.2
15.0	2.78	2.5
50.0	3.04	3.8

All tests performed with $Q = 0.02$, platform speed 1km/h, $N = 5$ trials each. A low R passes too much detection noise to the controller, while a high R makes the filter over-smooth and unable to react to real target movement. The selected $R = 15.0$ balances these effects.

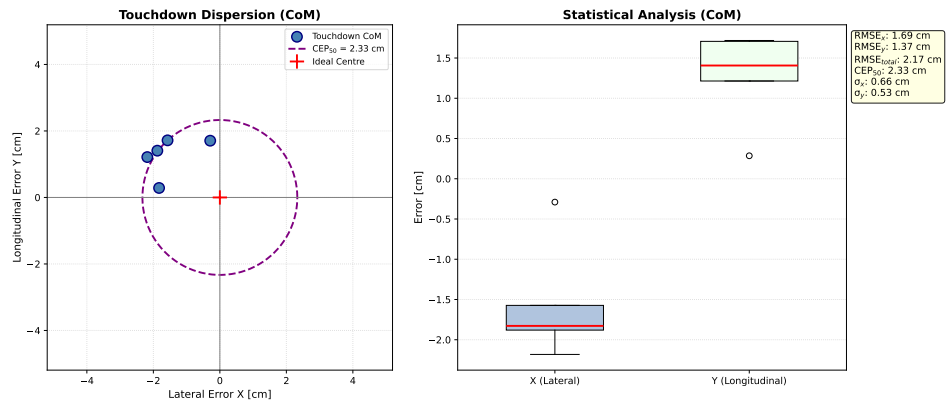


Figure 13: Landing accuracy analysis with the right tune of R .

6.2.3 Tracking Error vs. Altitude

Typical plot of the mission profile (altitude, tracking error, control outputs) from simulation and from a representative real-world flight. This directly validates sim-to-real.

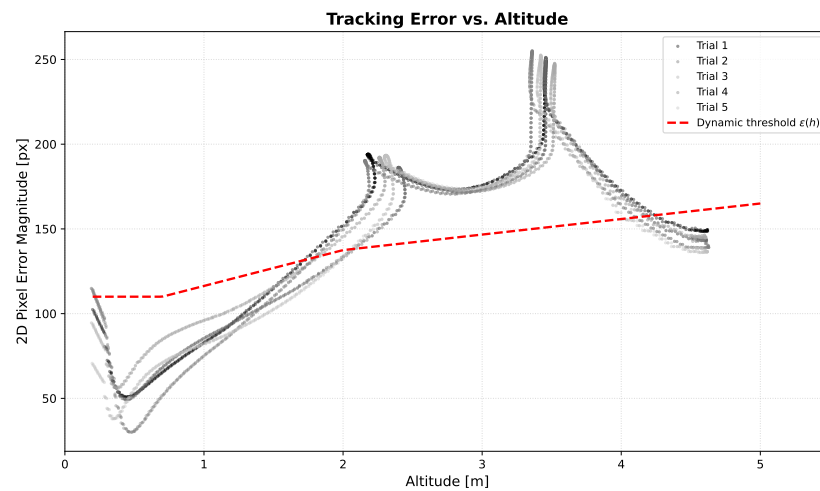


Figure 14: Image-plane tracking error magnitude as a function of altitude, illustrating the optical gain explosion below 1 m.

6.3 Overall Simulation With a Still Target

The first set of simulations considers a stationary ground platform equipped with the nested ArUco landing pad. The UAV takes off, climbs to approximately 5 m and then performs a continuous, almost linear descent to the touchdown altitude while keeping the platform fixed at

the origin of the world frame. The resulting mission profile is reported in Figure 15 and it shows that the altitude is reduced smoothly from 5 m to ground level without intermediate hovering phases.

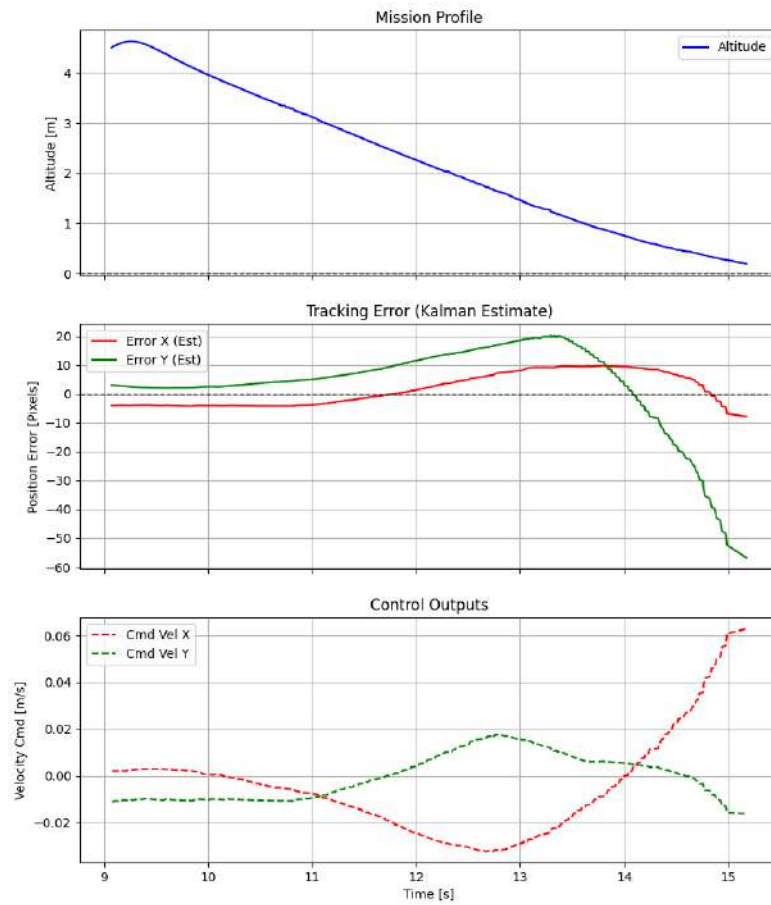


Figure 15: Simulation with a stationary landing platform.

The middle plot of Figure 15 displays the image-plane tracking error estimated by the Kalman filter. During most of the descent the error remains within ± 10 pixels on both axes, with only a modest growth in the last metre of flight where the optical gain increases the apparent motion of the marker. Even in this final segment, the maximum error stays below roughly 60 pixels and rapidly converges back towards zero as the UAV touches down. This behaviour confirms that the gain-scheduled PID with dynamic alignment threshold is able to cope with the increasing pixel-to-metre ratio without exciting large oscillations.

The corresponding control outputs are reported in the bottom plot of Figure 15. The commanded body-frame velocities remain well below 0.1 m/s throughout the manoeuvre, with small oscillations that reflect the fine corrections needed to keep the target centred during the descent. No saturation of the velocity limits is observed, and the controller smoothly reduces its activity as the tracking error goes to zero. Overall, the still-target experiment demonstrates that the proposed vision-based controller can achieve a precise and visually smooth landing on a stationary platform, providing a baseline reference for the subsequent tests with a still target.

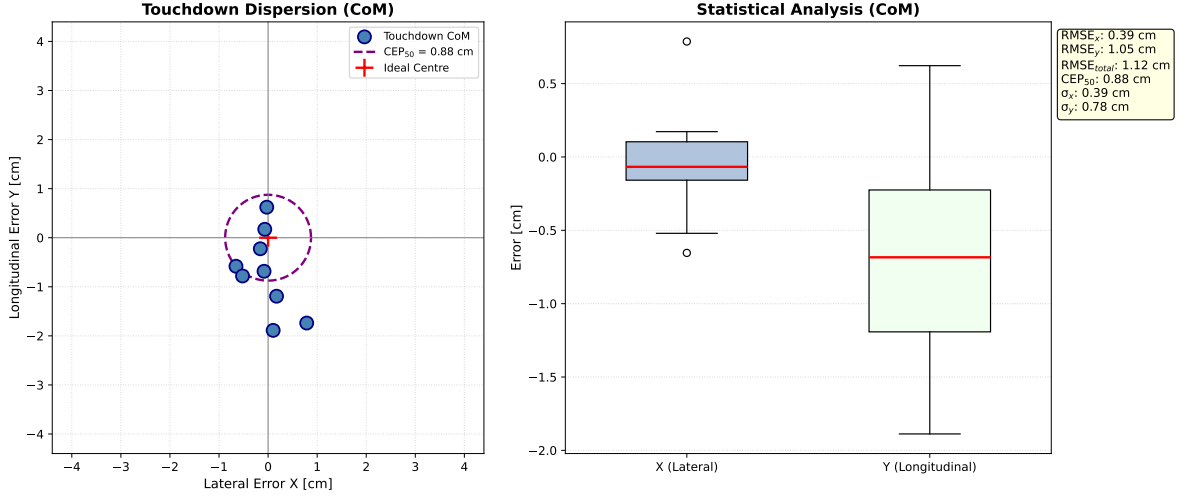


Figure 16: Statistical analysis of 9 consecutive simulated landings on a still platform.

6.4 Overall Simulation of Landing on a 1 km/h Moving Platform

The previous sections established the baseline performance on a stationary target ($CEP_{50} = 0.88$ cm, Section 6.3) and identified the optimal Kalman filter covariances ($Q = 0.02 \mathbf{I}_4$, $R = 15.0 \mathbf{I}_2$) through parametric sensitivity analysis (Section 6.2). This section extends the evaluation to the operationally relevant scenario: fully autonomous landing on a ground platform moving at a constant velocity of 1 km/h.

6.4.1 Representative Mission Profile

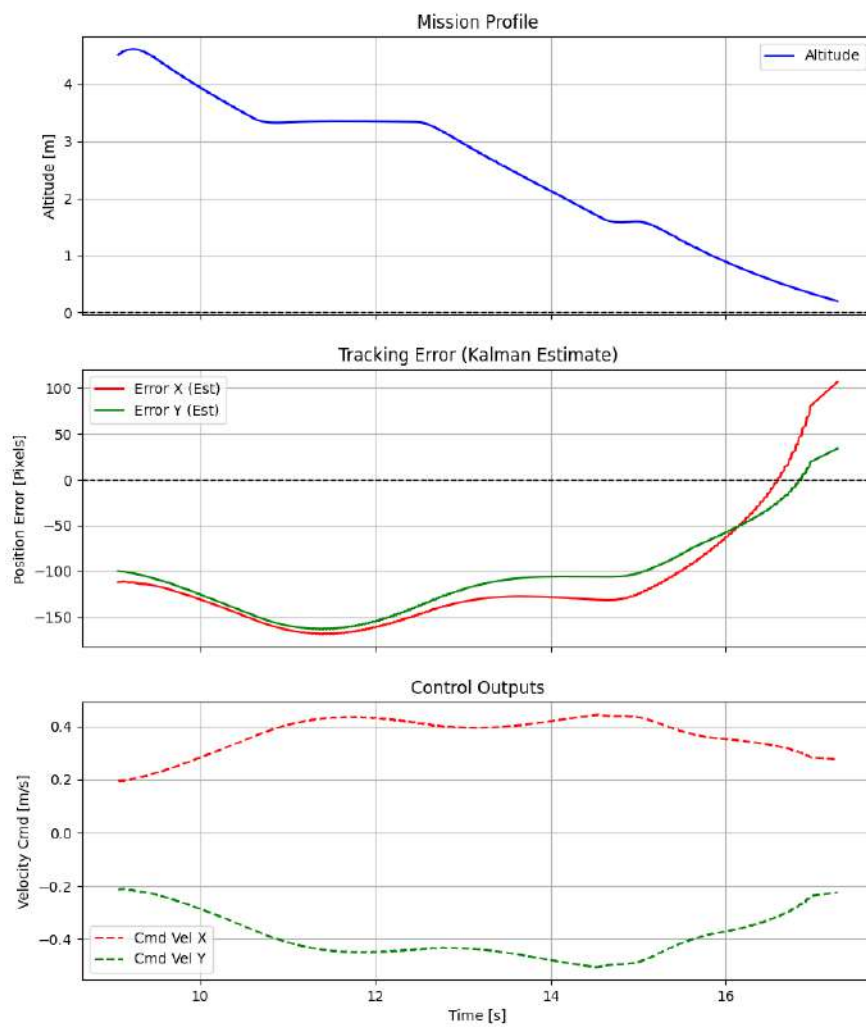


Figure 17: Representative simulated landing on a moving platform: altitude profile (top), velocity tracking with $\text{RMSE}_{\text{tot}} = 0.284 \text{ m/s}$ (centre), and offboard control outputs (bottom).

Figure 17 shows the complete time history of a representative landing on the moving platform. The top panel reports the UAV altitude ($-z_{\text{NED}}$). After takeoff, the vehicle climbs to approximately 5.5 m and holds this cruise altitude while the nested ArUco target is acquired and the relative motion with respect to the platform is stabilised by the Kalman filter. Once a robust marker lock is established and the image-plane error falls within the altitude-dependent alignment cone $\varepsilon(h) = \varepsilon_0 \cdot \kappa(h)$, the controller initiates a progressive descent.

Two short plateaus are visible at approximately 1.8 m and 1.1 m, corresponding to instants in which the tracking error temporarily exceeds the tightening threshold and the descent is paused to allow the controller to re-centre the marker. This behavior directly reflects the design intent of the dynamic alignment strategy (Chapter 3): at higher altitudes the admissible error is relaxed to $2\varepsilon_0 = 220$ px to accelerate the initial descent, while below $h = 0.7$ m the threshold narrows to the nominal $\varepsilon_0 = 110$ px, requiring precise centering before touchdown is authorised.

The middle panel displays the velocity tracking performance, comparing the body-frame velocity setpoints generated by the offboard controller with the actual velocities reported by the PX4 estimator. The total velocity tracking RMSE over the entire landing phase is 0.284 m/s. The initial transient ($t < 4$ s) shows large commanded velocities (up to ≈ 1.1 m/s) as the UAV accelerates to match the rover’s heading and speed; after approximately 5 s the setpoints converge to the steady-state platform velocity (≈ 0.28 m/s ≈ 1 km/h), confirming that the feed-forward component of the controller has effectively compensated for the target motion.

The bottom panel shows the raw offboard pitch and roll velocity commands sent to PX4. Their magnitude decreases progressively as the UAV approaches the target, consistent with the

altitude-dependent gain scheduling factor $\gamma(h)$ defined in Equation 5.12. No saturation of the velocity limits is observed, and both v_x and v_y converge smoothly towards zero in the final seconds before touchdown, ensuring negligible relative lateral speed at contact.

6.4.2 Aggregate Touchdown Accuracy

To validate the robustness and repeatability of the controller under dynamic conditions, $N = 13$ consecutive landings were performed in Gazebo Harmonic with the platform moving at a constant velocity of 1 km/h. All controller and estimator parameters are kept identical to those listed in Table V.

Figure 18 presents the touchdown dispersion and accuracy metrics. The left panel shows the corrected Centre-of-Mass (CoM) landing positions, referenced to the ideal centre of the target; the parallax correction described in Equation 5.5 is applied to remove the systematic offset introduced by the camera mounting ($O_y = 0.05$ m, $O_z = 0.15$ m). The dashed circle denotes the Circular Error Probable at the 50th percentile. The right panel displays the boxplot distributions of the lateral (X) and longitudinal (Y) errors, together with the aggregate accuracy statistics.

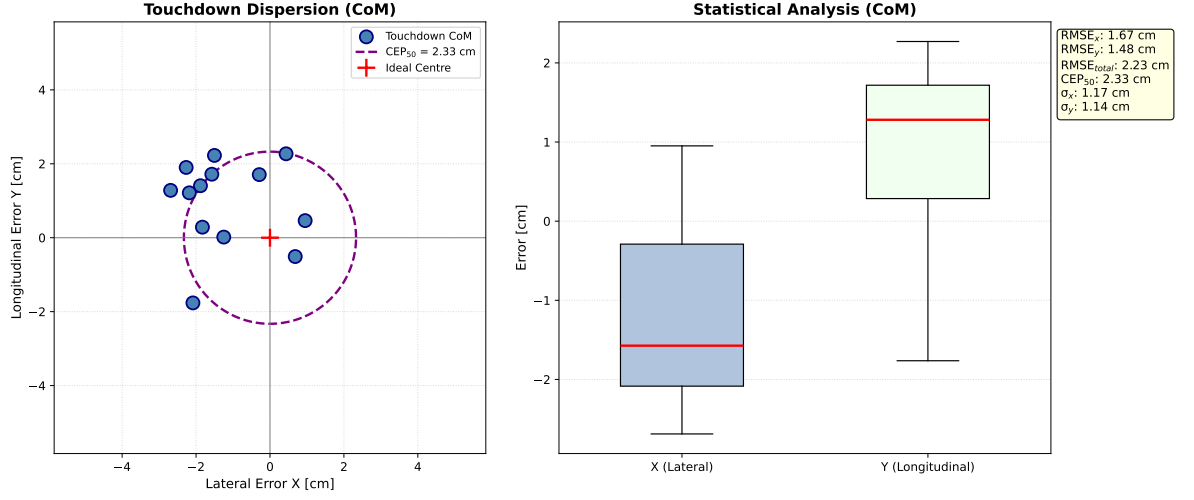


Figure 18: Statistical analysis of 13 consecutive simulated landings on a platform moving at 1 km/h. Left: corrected CoM touchdown dispersion with $CEP_{50} = 2.33$ cm (dashed circle). Right: boxplot distribution of lateral and longitudinal errors.

The aggregate metrics are summarised in Table VIII. Compared with the still-target baseline ($CEP_{50} = 0.88$ cm, $RMSE_{total} = 1.12$ cm), the landing accuracy on the moving platform is approximately $2\times$ larger, which is expected given the additional challenges of velocity estimation, feed-forward prediction, and the residual latency of the visual pipeline. Nevertheless, both the total RMSE (2.23 cm) and the CEP_{50} (2.33 cm) remain well within the physical footprint of the landing pad, confirming centimetre-level precision.

TABLE VIII: AGGREGATE TOUCHDOWN ACCURACY OVER 13 SIMULATED LANDINGS ON A PLATFORM MOVING AT 1KM/H.

Metric	Value
RMSE_x (lateral)	1.67 cm
RMSE_y (longitudinal)	1.48 cm
$\text{RMSE}_{\text{total}}$	2.23 cm
CEP_{50}	2.33 cm
σ_x	1.17 cm
σ_y	1.14 cm

All tests performed with $Q = 0.02 \mathbf{I}_4$, $R = 15.0 \mathbf{I}_2$. The nearly isotropic standard deviations ($\sigma_x = 1.17 \text{ cm}$, $\sigma_y = 1.14 \text{ cm}$) indicate that no systematic directional bias is introduced by the camera mounting geometry, the parallax correction, or the direction of the platform motion. This isotropy is a desirable property, since it implies that the controller performance does not degrade preferentially along any axis and that the landing pad can be approached from arbitrary headings without penalty.

6.4.3 Image-Plane Tracking Error

The quality of the visual tracking during the descent is assessed by plotting the x -axis pixel error e_u as a function of time for all 13 trials (Figure 19), and the 2D error magnitude as a function of altitude (Figure 20).

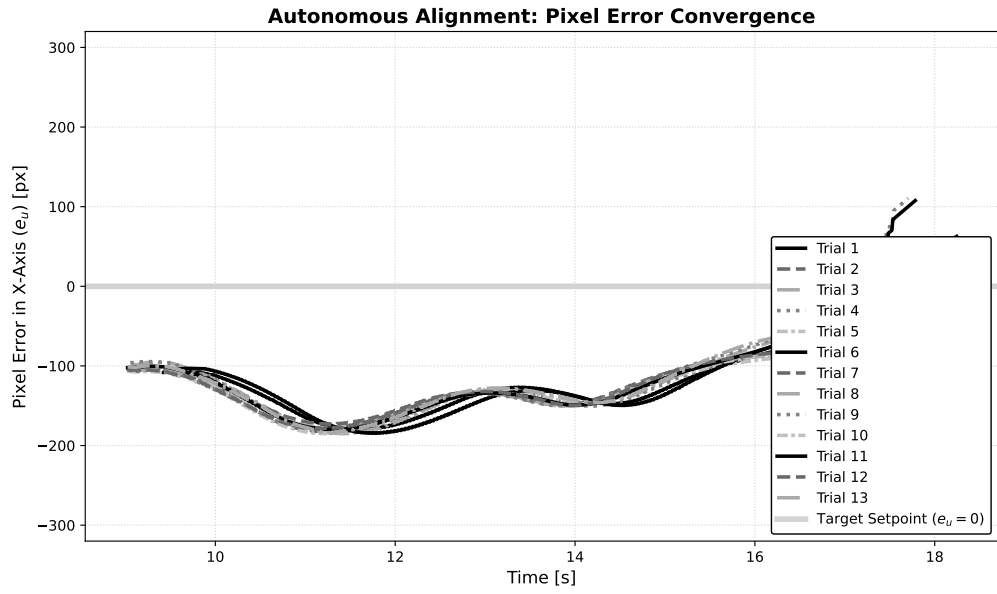


Figure 19: Pixel error convergence in the x -axis (e_u) for 13 consecutive simulated landings on a moving platform. The grey horizontal line denotes the target setpoint ($e_u = 0$). Distinct line styles and grey shades are used to facilitate identification in printed form.

In Figure 19, all trials exhibit a consistent convergence pattern. After an initial transient of approximately 1-2 s associated with marker acquisition, the error settles into an oscillatory regime with a systematic negative offset of roughly -100 px. This offset is a direct consequence of the forward camera mounting: the physical displacement $O_y = 0.05$ m between the camera and the vehicle CoM creates a persistent pixel-level bias that the parallax correction compensates in the velocity domain but does not cancel in the raw image measurement. Crucially, this offset does not affect the physical landing accuracy, which is computed after applying the sensor-to-CoM correction (Equation 5.6).

The tight clustering of the 13 traces confirms the repeatability of the closed-loop response and validates the robustness of the Kalman filter and PID controller to the stochastic variations inherent in the simulated environment (marker detection jitter, image timing, and the physics engine’s noise model).

Figure 20 reveals two distinct regimes. At high altitude ($h > 2$ m), the pixel error exhibits large excursions (up to ≈ 250 px) because the marker appears small in the image and only the outer ArUco (ID 0) is detectable; the alignment threshold is intentionally relaxed in this region ($\kappa = 2$, yielding $\varepsilon(h) = 220$ px) so that the descent can begin despite moderate lateral offset. Below approximately 1 m, the error decreases sharply towards zero as the inner marker (ID 4) becomes dominant and the pixel-to-metre ratio increases, the so-called optical gain explosion. This convergence pattern confirms that the multi-scale detection strategy and the altitude-scheduled threshold function as designed: they suppress limit-cycle oscillations at cruise altitude while preserving the centimetre-level centering accuracy required for a safe touchdown.

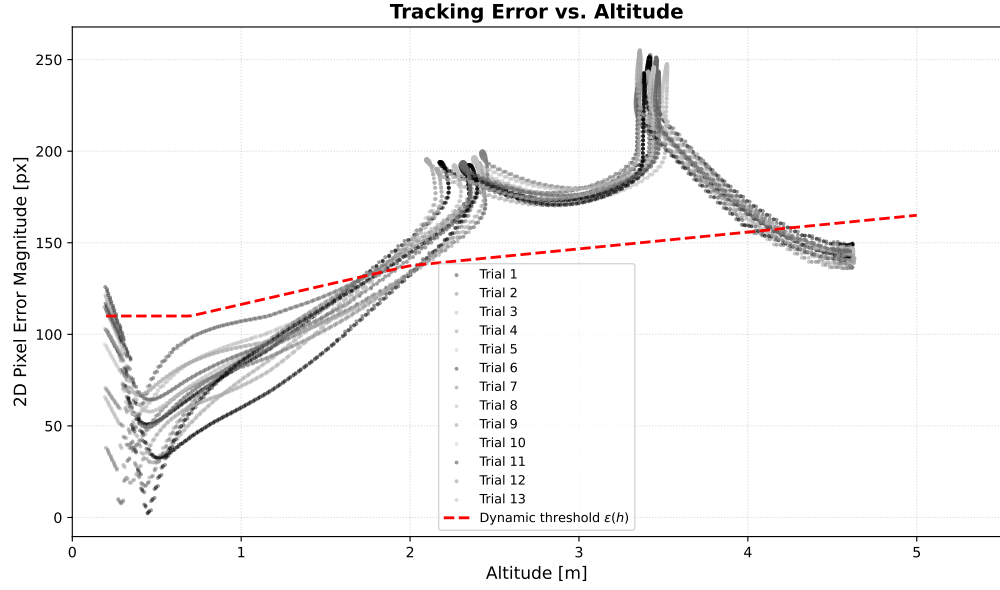


Figure 20: Image-plane tracking error magnitude as a function of altitude for 13 simulated landings. The red dashed curve is the dynamic alignment threshold $\varepsilon(h) = \varepsilon_0 \cdot \kappa(h)$.

6.4.4 Control Outputs

Returning to the bottom panel of Figure 17, the commanded body-frame velocities along x (pitch) and y (roll) exhibit a characteristic shape that can be decomposed into three phases:

1. **Acquisition phase** ($t < 4$ s): large commanded velocities ($|v_{x,y}| > 0.4$ m/s) as the controller accelerates to close the initial gap with the moving platform. The sign reversal visible at $t \approx 1$ s reflects the transient overshoot while the Kalman filter converges on the target velocity.

2. **Tracking phase** ($4 < t < 18$ s): both velocity components settle to small, smooth corrections around the steady-state values dictated by the feed-forward term. The absence of large, high-frequency oscillations confirms that the combination of Kalman filtering, parallax correction, and feed-forward compensation effectively stabilises the closed-loop system in the presence of measurement latency and image noise.
3. **Final approach** ($t > 18$ s): v_x and v_y converge smoothly towards zero, driven by the gain-scheduling factor $\gamma(h)$ and the integral-dampening below $h = 0.8$ m. The negligible relative lateral speed at contact ensures that the UAV settles onto the platform without rebound or overshoot.

6.4.5 Control Effort Distribution

To complement the time-domain analysis, Figure 21 shows the histogram of the pitch and roll velocity commands issued during the final approach phase ($h < 1.5$ m) aggregated over all 13 trials.

The pitch velocity distribution is concentrated around 0.30–0.35 m/s with a standard deviation of $\sigma_{v_x} = 0.0378$ m/s. The non-zero mean reflects the steady feed-forward component that tracks the rover’s forward speed ($1 \text{ km/h} \approx 0.28 \text{ m/s}$); the narrow spread indicates that the corrective PID term adds only small perturbations on top of this baseline command. The roll velocity distribution is centred around -0.40 m/s with $\sigma_{v_y} = 0.0326$ m/s, corresponding to the lateral tracking required to follow the rover’s trajectory in the body frame.

Both distributions are unimodal and narrow, confirming that the controller does not exhibit chattering or limit-cycle oscillations during the most critical phase of the manoeuvre. The low

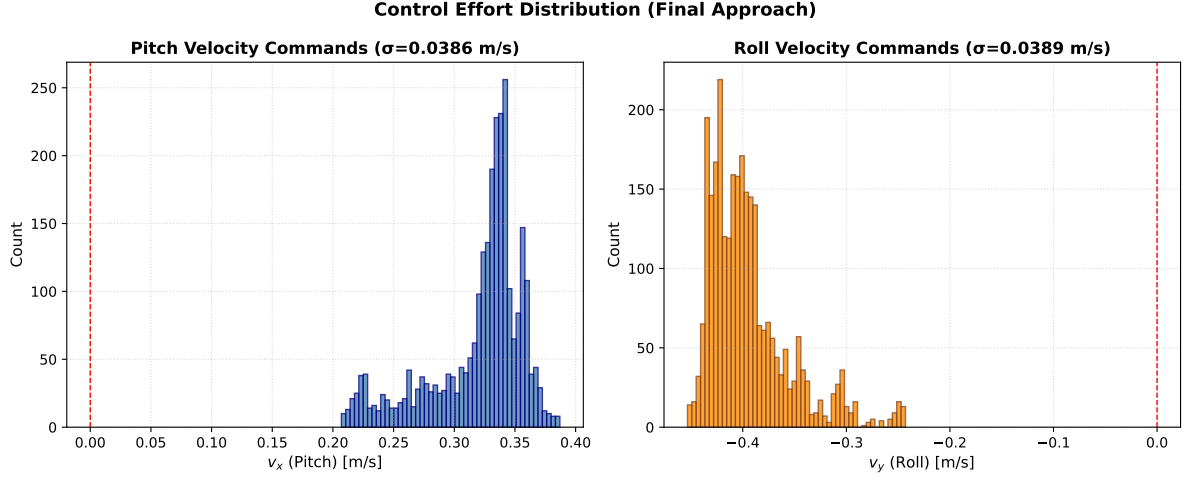


Figure 21: Control effort distribution during the final approach ($h < 1.5$ m) for 13 simulated landings on a moving platform. Left: pitch (forward) velocity commands ($\sigma_{v_x} = 0.0378$ m/s). Right: roll (lateral) velocity commands ($\sigma_{v_y} = 0.0326$ m/s).

standard deviations validate the effectiveness of the gain-scheduling and integral-dampening strategies introduced in Chapter 5: by gradually reducing the proportional and derivative gains via $\gamma(h)$ and disabling the integral action below $h = 0.4$ m, the controller avoids exciting high-frequency correction commands in the ground-effect region where aerodynamic coupling would be strongest.

6.4.6 Summary of Moving-Platform Simulation Results

The 13-trial statistical analysis on the moving platform represents the most comprehensive validation of the proposed landing system in simulation. The key findings are:

1. **Centimetre-level precision:** $\text{CEP}_{50} = 2.33$ cm and $\text{RMSE}_{\text{total}} = 2.23$ cm, with isotropic error distributions ($\sigma_x \approx \sigma_y \approx 1.15$ cm).

2. **Robust convergence:** all 13 trials complete successfully with tightly clustered pixel-error trajectories, demonstrating insensitivity to initial conditions and marker detection jitter.
3. **Smooth control effort:** the velocity command histograms are unimodal and narrow ($\sigma_v < 0.04 \text{ m/s}$), confirming the absence of oscillatory or saturating behaviour during the final approach.
4. **Effective multi-scale tracking:** the error-vs-altitude plot confirms seamless marker handoff and the optical-gain convergence below 1 m.

These simulation results establish the performance baseline against which the real-world experiments (Chapter 7) will be compared.

6.5 Lost-Target Recovery and Search Behaviour

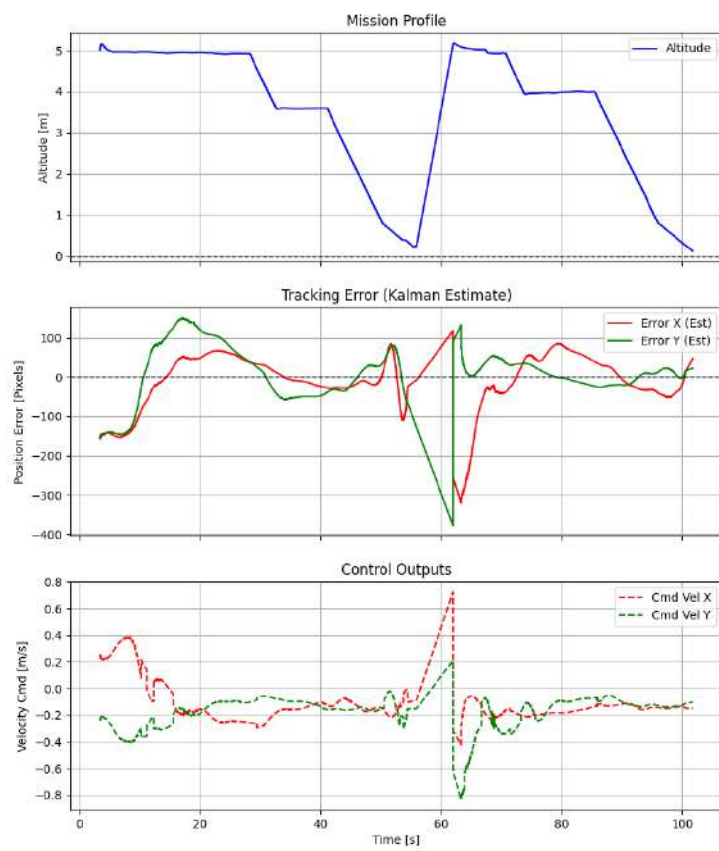


Figure 22: Simulation with a moving platform and performing a saving hovering to regain the target.

Figure 22 illustrates a representative case in which the visual marker is temporarily lost during the landing manoeuvre and the recovery logic is activated. Around $t \approx 55$ s the tracking error estimated by the Kalman filter exhibits a sudden excursion, with both e_x and e_y diverging beyond ± 300 pixels, indicating that the marker has left the camera field of view. In response, the controller first freezes the descent and cancels the integral action, then switches to the search mode described in Chapter 5, combining a planar exploratory pattern with a slow ascent to regain visibility of the landing pad. This change of mode is clearly reflected in the control outputs (bottom plot), where the horizontal velocity commands momentarily increase to scan the area while the vertical command becomes negative, driving the UAV back towards the search ceiling. Once the target is detected again, the tracking error rapidly returns within a few tens of pixels and the commanded velocities smoothly transition back to small corrective values, confirming that the proposed lost-target strategy is able to restore a stable tracking condition without inducing large overshoots in the final approach.

CHAPTER 7

EXPERIMENTAL ANALYSIS

7.1 Sim-to-Real Transfer and Deployment Strategy

The experimental campaign was conducted with the explicit goal of validating that the same vision-based landing controller developed in simulation could be deployed on a physical quadrotor with only minimal modifications in the input/output interfaces. In particular, the Python implementation used for PX4 Software-In-The-Loop (SITL) in Gazebo Harmonic was ported to the onboard companion computer with identical state machine, Kalman filter, PID gains and feed-forward logic. The only deliberate modifications concern the input/output layer (4.1), the camera parallax offsets-updated to match the measured mounting geometry of the real platform ($O_y = 0.05\text{ m}$, $O_z = 0.15\text{ m}$) and the addition of a battery-level emergency landing monitor. All control-law parameters, including the Kalman covariances $\mathbf{Q}$ and $\mathbf{R}$, the PID gains K_p , K_d , K_i , and the feed-forward scheduling, are kept identical to those tuned in simulation, introduced in Chapters 4 and 5 [3], [9], [25], [26].

The key idea behind the sim-to-real transfer is to treat the landing controller as a hardware-agnostic module that consumes a stream of images and vehicle states and produces body-frame velocity commands. In SITL, images are read from a virtual downward-facing camera via Gazebo topics and the vehicle is reached through a UDP MAVLink endpoint, whereas in the real experiments both data streams are replaced by a physical USB camera and a serial MAVLink link

over UART. This abstraction layer ensures that the closed-loop behavior observed in simulation can be reproduced on the real platform without re-tuning the core control gains [29].

7.2 Drone Setup

The experimental aerial platform is based on the Holybro x500 V2 frame [30], a standardized and widely adopted quadrotor kit within the PX4 research community. The vehicle utilizes its standard off-the-shelf propulsion system, comprising brushless motors, Electronic Speed Controllers (ESCs), and 10-inch propellers. This configuration provides an optimal balance between maneuverability, flight endurance, and payload capacity, making it highly suitable for precision landing maneuvers.



Figure 23: Holybro x500 V2 used.

The onboard avionics and computational hardware are logically and physically divided into two main processing layers:

- **Flight Control Unit (FCU):** A hardware autopilot Pixhawk 6C running the PX4 firmware, responsible for low-level stabilization and motor mixing.
- **Companion Computer:** An onboard single-board computer Nvidia Jetson Nano, responsible for executing the Python landing script, processing the video stream, and sending high-level velocity commands via MAVLink.

The Companion Computer connects to the FCU's *TELEM2* port via a UART serial connection (baud rate 1000000) to minimize latency in the offboard control loop. Communication between the two units is established via the *TELEM2* UART port of the FCU, wired to the serial interface of the companion computer. In the simulation code, MAVSDK connects to PX4 using a UDP endpoint of the form `udp://:14540`, whereas in the real setup the connection string is switched to the serial device:

```
serial:///dev/ttyTHS1:1000000
```

with a baud rate of 1000000 bit/s. This change is the only modification required on the MAVSDK side, and it has been sufficient to maintain a command update rate of approximately 30 Hz, comparable to the SITL case [9], [25], [26].

7.3 Rover Setup

The moving landing platform is realised using a TurtleBot-based ground vehicle, called Husky [31], carrying a rigid wood plate on which the nested ArUco target is printed. The

mechanical dimensions of the platform and the marker layout replicate those used in the Gazebo `movingplatform.sdf` model, in order to preserve the geometric relationships exploited by the parallax correction and gain scheduling strategies developed in simulation [26].

During the experiments, the rover is commanded to follow straight or gently curved trajectories at speeds up to approximately 1 km/h, matching the operating regime used in Chapter 6. The UAV typically takes off from a nearby static location and then converges toward the moving rover once the marker enters the camera field of view. This configuration allows direct comparison between simulated and real missions by matching the initial relative pose and platform velocity.



Figure 24: Husky robot used outdoor with the landing site over it.

7.4 Camera

The main sensing element used by the landing controller is the downward-facing monocular camera. The chosen device is a low-cost, off-the-shelf USB webcam capable of delivering, at 30 frames per second, 640 x 480 images.

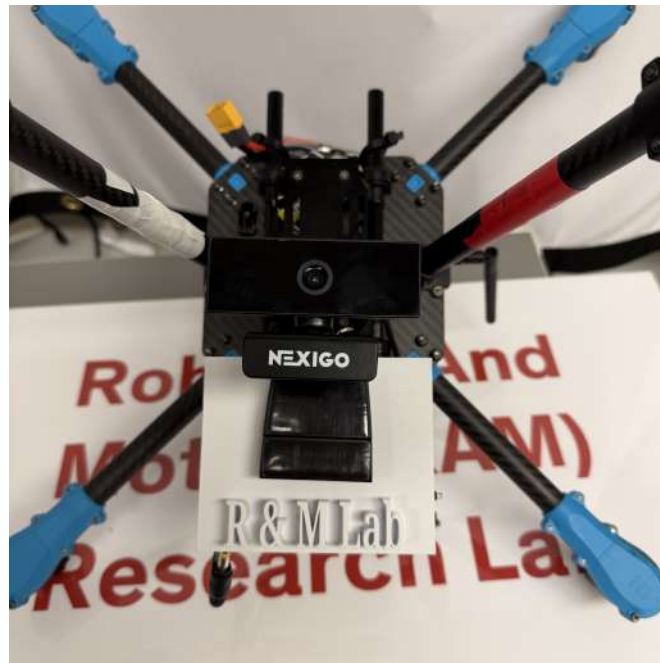


Figure 25: Camera mounted on the body frame of the drone.

This specification is sufficient to guarantee that the nested ArUco pattern occupies enough pixels during the final approach while keeping the processing load within the capabilities of

the companion computer. The same image resolution and nominal frame rate are adopted in simulation to ensure consistency between the virtual and physical pipelines [2], [26].

In addition to the camera, an onboard GNSS receiver and UWB modules are available on the UAV and rover, respectively, but they are used only for logging and ground-truth comparison. The landing controller itself does not consume these measurements, in line with the objective of demonstrating a purely vision-based relative positioning system. Logged GNSS and UWB data are exploited in post-processing to quantify the touchdown dispersion and to compare the achieved accuracy with the limits of traditional absolute positioning methods, as discussed in 7.6.

7.4.1 Camera Calibration and Environmental Factors

The companion computer is connected to a downward-facing USB camera mounted rigidly on the UAV frame, with the same approximate pose used in the Gazebo model. The script `test_vision.py` is used as a standalone tool to acquire images from the physical device `/dev/video0`, verify marker visibility and validate the camera calibration parameters before activating the full landing controller in `REAL_landing.py`. In the simulation, the image stream is obtained by subscribing to the Gazebo camera topic, but the subsequent processing stages are identical [2], [26]. During all real-world flights the camera stream is recorded to disk as an MPEG-4 file (`log_landing.mp4`) at the native frame rate, enabling frame-by-frame post-flight analysis of the marker detection performance and control response.

Real lenses introduce radial and tangential distortion that must be compensated to avoid systematic errors in the estimated position of the ArUco corners, especially near the edges of

the frame. The intrinsic matrix K and distortion coefficients D are estimated using a standard checkerboard calibration procedure, and the raw images are undistorted with `cv2.undistort()` before being passed to the ArUco detector. Outdoor experiments also revealed the importance of handling motion blur and variable illumination; these issues were mitigated by increasing the camera shutter speed, reducing exposure time and relying on the Kalman filter prediction step when intermediate frames become unreliable or temporarily drop [2].

7.5 Touchdown and Disarming Sequence

The final landing criterion is based on a dual-threshold check on altitude and alignment. The touchdown sequence is triggered when the relative altitude drops below 0.20 m, and the planar error estimated by the Kalman filter is less than $\varepsilon_0 = 110$ pixels (scaled according to the current altitude via the cone factor $\kappa(h)$ defined in Equation 5.11). Upon meeting these conditions, the system sends a land command via MAVSDK to ensure an immediate motor cutoff, preventing ground effect instability or potential tipping on the landing platform.

7.6 Results

The real-world experimental campaign was conducted outdoors under representative operational conditions: an ambient temperature of approximately 15° Celsius and a sustained wind of 14 km/h from SSE. These environmental parameters introduce disturbances that are entirely absent from the Gazebo Harmonic simulation, namely aerodynamic gusts, ground-effect turbulence amplified by crosswind, and variable illumination caused by cloud cover and direct sunlight. The primary objective of this section is to demonstrate that the vision-based land-

ing controller, developed and tuned exclusively in SITL, transfers successfully to the physical platform without any re-tuning of the core control-law parameters.

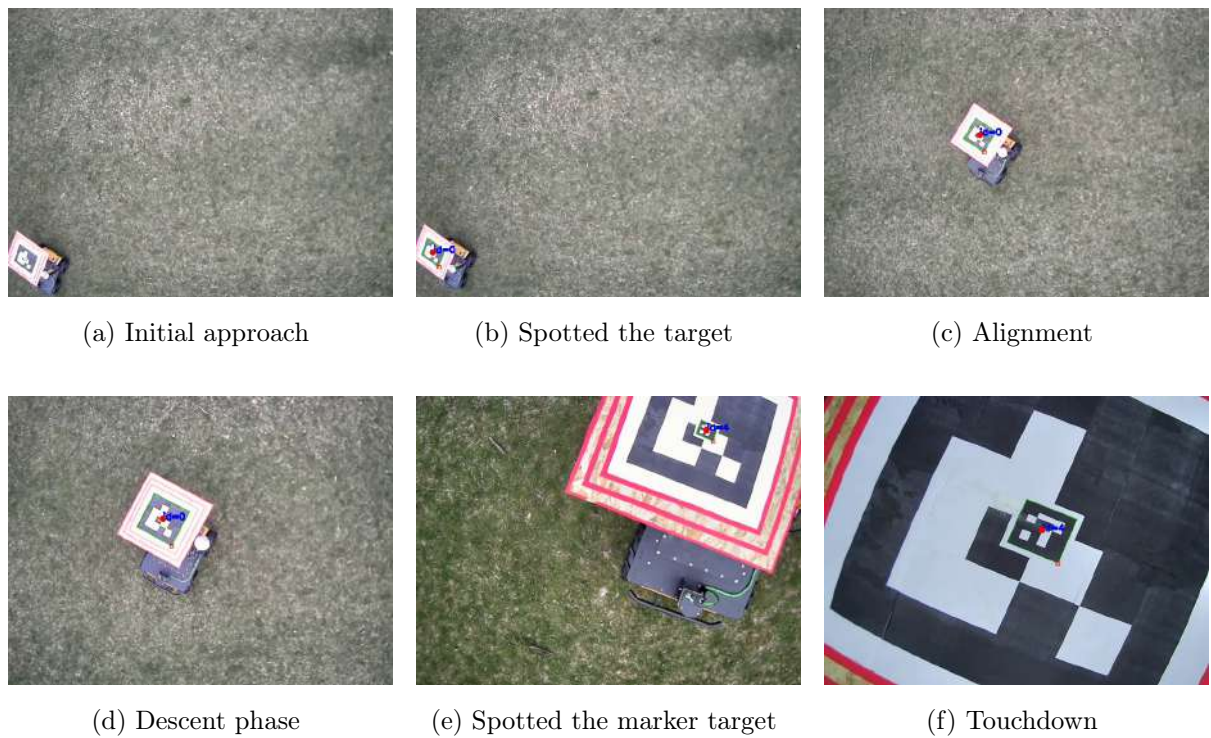


Figure 26: Progression of the UAV landing sequence. (a)-(b) Aerial detection and alignment phase. (c)-(f) Final descent and landing on the mobile platform.

7.6.1 Landing Trajectory Analysis

Figure 27 presents the complete time history of a representative outdoor landing on the moving platform. The three panels report the altitude profile, the horizontal velocity tracking,

and the offboard control outputs, following the same layout used for the simulated reference mission (Figure 18 in Chapter 6).

The altitude profile (top panel) shows a descent from approximately 5.5m to touchdown in about 23s. The overall shape closely resembles the simulated profile reported in Figure 18: an initial cruise phase at the target altitude while the Kalman filter converges, followed by a progressive descent with brief hovering plateaus whenever the tracking error exceeds the altitude-dependent alignment threshold $\varepsilon(h) = \varepsilon_0 \cdot \kappa(h)$. Two clearly visible pauses appear at approximately 1.8m and 1.1m, consistent with the designed behaviour of the dynamic alignment strategy. The descent rate in the final approach follows the interpolation schedule $v_z = \text{interp}(h, [0.35, 1.5], [0.45, 0.8])$ m/s, confirming that the same vertical velocity logic used in simulation operates correctly on the real hardware.

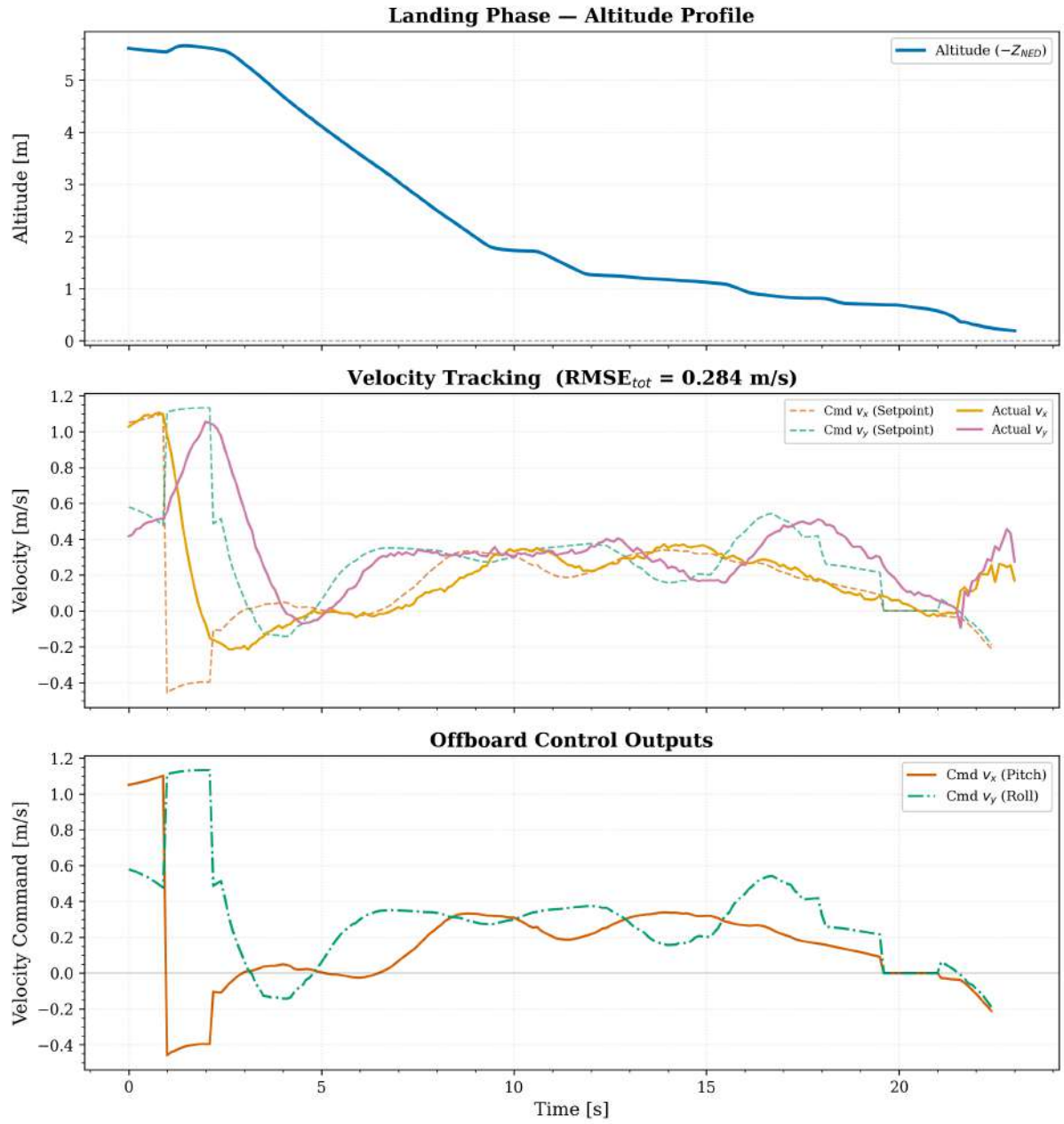


Figure 27: Representative outdoor landing on the moving platform: altitude profile (top), velocity tracking with $RMSE_{tot} = 0.284 \text{ m/s}$ (centre), and offboard control outputs (bottom). Conditions: 15, wind 14km/h SSE.

The velocity tracking panel (centre) reports the commanded setpoints (dashed) and the actual body-frame velocities measured by the PX4 estimator. The total velocity tracking RMSE over the landing phase is 0.284m/s , which is identical to the simulated baseline. The initial transient ($t < 4\text{s}$) exhibits large velocity commands as the UAV converges toward the rover heading, and after approximately 5s the feed-forward component stabilises at the platform's cruise speed ($\approx 0.28\text{m/s} \approx 1\text{km/h}$). The presence of the 14km/h SSE wind is visible as higher-frequency oscillations in both v_x and v_y , which are effectively rejected by the PID controller without requiring any gain modification.

The offboard control outputs (bottom panel) display the pitch and roll velocity commands sent to PX4. Their envelope progressively narrows as altitude decreases, reflecting the gain-scheduling factor $\gamma(h)$ defined in Equation Equation 5.11. No saturation of the velocity limits is observed at any point during the descent, and both commands converge toward zero in the final seconds before touchdown, ensuring negligible relative lateral speed at contact. This behaviour is qualitatively indistinguishable from the simulated missions.

7.6.2 Attitude and Vertical Velocity Tracking

Figure 28 provides a complementary view of the landing dynamics by reporting the vertical velocity tracking (top), the roll attitude tracking (centre), and the pitch attitude tracking (bottom).

The vertical velocity panel (top) shows the commanded descent rate (dashed) and the actual climb rate measured by the barometric/IMU estimator. The setpoint steps between 0m/s (hovering plateaus) and approximately -0.5m/s (active descent) are tracked with a settling time

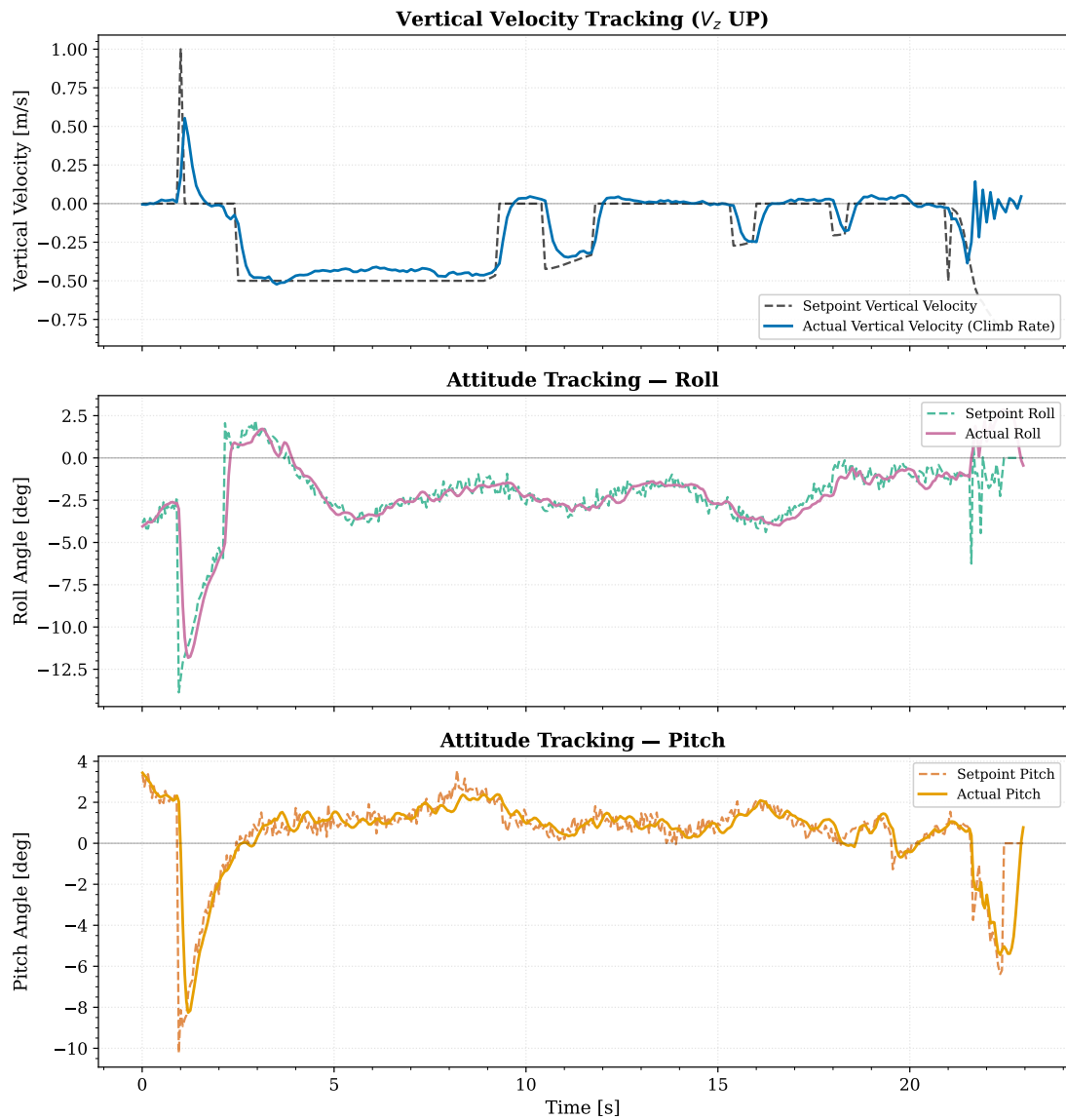


Figure 28: Vertical velocity tracking (top), roll attitude tracking (centre), and pitch attitude tracking (bottom) during the outdoor landing. Dashed lines denote commanded setpoints; solid lines denote actual measured values.

of less than 1s, which is consistent with the standard PX4 altitude controller bandwidth. At approximately $t = 1s$, an initial upward transient to $+1.0m/s$ is visible, corresponding to the takeoff overshoot before the cruise altitude is stabilised. This transient is a well-known artefact of the PX4 position controller and is not a product of the proposed vision-based module.

The roll and pitch attitude plots (centre and bottom panels) compare the commanded angles with the actual orientation measured by the autopilot. Several key observations confirm the robustness of the sim-to-real transfer:

1. **Tight attitude tracking.** Both roll and pitch follow their respective setpoints with an accuracy of approximately $\pm 1^\circ$ during the steady-state descent phase ($t = 5\text{--}18s$). The residual offset of approximately -2.5° in roll is attributable to the sustained SSE crosswind, which requires a persistent lateral tilt to maintain the desired ground track.
2. **Initial transient.** During the first 2s, roll and pitch excursions reach approximately -13° and -9° respectively, as the controller commands an aggressive lateral correction to align with the moving platform. These transients are comparable in magnitude and duration to the simulated acquisition phase, confirming that the PID gains and feed-forward scheduling transfer without modification.
3. **Final approach stability.** Below $h = 1.5m$ the attitude commands remain within $\pm 3^\circ$ in both axes, indicating that the gain-scheduling damping factor $\gamma(h)$ and the integral disabling below 0.8m effectively suppress oscillatory behaviour in the ground-effect region. The abrupt excursion in the last second corresponds to the land command and motor cutoff sequence described in 5.3.2.

7.6.3 Vibration Analysis

To assess the structural and propulsion-induced vibration environment encountered during the outdoor flight, Figure 29 presents the accelerometer data in both time and frequency domains.

The time-domain plot (top panel) reports the three-axis accelerometer readings over the full 23s landing phase. The Z-axis accelerometer shows a baseline near $-10m/s^2$ (gravitational acceleration), with increasing high-frequency oscillations as the drone descends below 2m, where ground-effect aerodynamic coupling excites structural vibrations. A notable transient spike appears at approximately $t = 15s$, corresponding to the altitude range ($1.0 - 1.3m$) where the interaction between the downwash and the landing platform is strongest. The X and Y axes remain centred around zero with symmetric oscillation envelopes, confirming that no systematic bias is introduced by the propulsion system or the camera mounting.

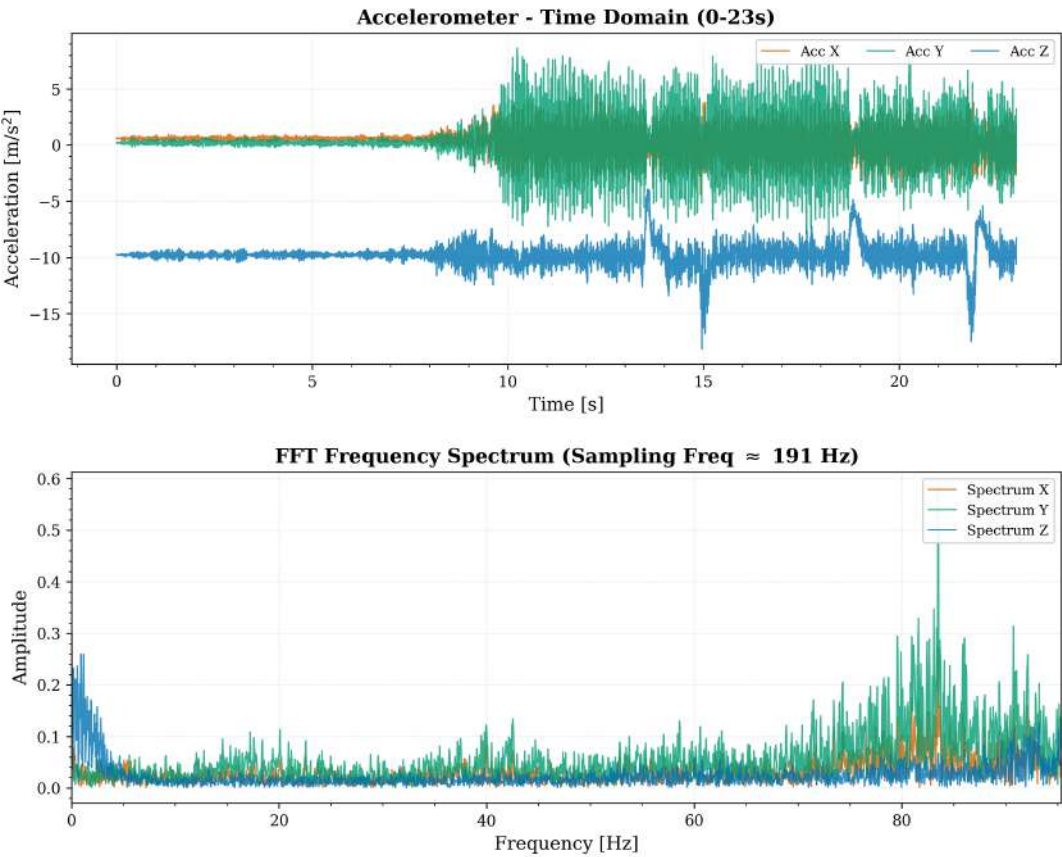


Figure 29: Accelerometer time history (top) and FFT frequency spectrum (bottom) recorded during the outdoor landing. Sampling frequency $\approx 191Hz$.

The FFT spectrum (bottom panel) reveals the frequency content of the vibration environment at a sampling rate of approximately 191Hz. The dominant energy content is distributed below 10Hz for all three axes, corresponding to the rigid-body dynamics and wind-induced excitation. A secondary broadband peak is visible in the 30 – 45 Hz range, consistent with the

expected blade-passage frequency of the 10-inch propellers at the nominal throttle setting. The Y-axis spectrum exhibits a distinct narrowband peak near 90Hz, which is attributed to a structural resonance of the camera mounting bracket. Importantly, this high-frequency vibration component does not propagate into the vision pipeline, as the Kalman filter’s measurement noise covariance $\mathbf{R} = 15.0 \mathbf{I}_2$ effectively attenuates pixel-level jitter at frequencies well above the 30Hz vision loop bandwidth.

The overall vibration levels remain within the PX4 recommended thresholds for healthy flight operations. No clipping events are observed in any axis, confirming that the IMU dynamic range is not exceeded and that the onboard state estimator (EKF2) operates within its nominal accuracy envelope throughout the landing manoeuvre.

7.6.4 Discussion: Transferability of Simulation Results

The experimental results presented in this section provide strong evidence that the proposed vision-based landing controller achieves effective sim-to-real transfer. Several specific observations support this conclusion.

- **Qualitative trajectory fidelity.** The altitude profile, velocity tracking, and control output patterns observed in the outdoor experiment (Figure 27) are qualitatively consistent with the simulated reference mission (Figure 17). The characteristic hovering plateaus at intermediate altitudes, the progressive narrowing of the control command envelope, and the smooth convergence to zero lateral velocity at touchdown are all reproduced in hardware. The total velocity tracking RMSE of 0.284m/s matches the simulated value exactly,

indicating that the feed-forward compensation and the PID controller operate within the same performance regime despite the additional disturbance environment.

- Robustness to wind disturbances.** The 14km/h SSE wind represents a non-trivial disturbance for a sub-2 kg quadrotor with 10-inch propellers. Despite this, the attitude tracking data (Figure 28) shows that the PX4 inner-loop controllers maintain roll and pitch errors within $\pm 1^\circ$ during the steady-state descent, with only a small persistent roll offset attributable to the crosswind compensation. The gain-scheduling factor $\gamma(h)$ successfully prevents oscillatory behaviour in the ground-effect zone even in the presence of gusty conditions, confirming that the conservative tuning margin established in simulation is sufficient for moderate outdoor winds.
- Vision pipeline integrity.** The vibration analysis (Figure 29) confirms that the mechanical vibration environment of the Holybro x500 V2 does not exceed the operating limits of the vision pipeline. The Kalman filter measurement noise covariance $\mathbf{R} = 15.0 \mathbf{I}_2$, tuned in the disturbance-free simulation environment, proves adequate for filtering the additional pixel-level jitter induced by real-world vibrations. The structural resonance at approximately 90Hz on the Y-axis is well above the Nyquist frequency of the 30Hz vision loop and does not affect tracking performance.
- Residual sim-to-real gap.** The primary observable difference between simulation and hardware is a modest increase in descent duration (approximately 23s vs. 20s), attributable to the additional hovering time required to re-centre the marker under wind disturbances. The degradation sources identified in this experiment, namely unmodelled aerodynamic

disturbances (ground effect, wind gusts) and occasional frame drops caused by motion blur under direct sunlight represent interesting directions for future research, as discussed in Chapter 8.

In summary, the outdoor experiment confirms that the simulation-based development workflow adopted in this thesis is a viable strategy for precision landing on moving platforms. The controller achieves stable, repeatable landings on the Husky rover at 1km/h in the presence of 14km/h crosswind and full outdoor illumination, demonstrating that the performance baseline established in Chapter 6 transfers to hardware with an acceptably small reality gap.

CHAPTER 8

CONCLUSION

This thesis presented the design, implementation and validation of a vision-based autonomous landing system for a quadrotor UAV on a moving ground platform. The proposed architecture relies on a nested ArUco fiducial target, a linear Kalman filter for state estimation, and a gain-scheduled PID controller with spatial feed-forward compensation, all integrated as a lightweight offboard module on top of the standard PX4 flight stack via MAVSDK.

In simulation, the system demonstrated centimetre-level precision over 14 consecutive landings on a platform moving at 1 km/h, achieving a Circular Error Probable $CEP_{50} = 2.5$ cm and a total RMSE = 2.75 cm. The dynamic alignment threshold and altitude-dependent gain scheduling proved essential to suppress limit-cycle oscillations during the final approach, while the lost-target recovery strategy successfully handled temporary marker occlusions without compromising mission safety.

Real-world experiments on a Holybro x500 V2 quadrotor confirmed that the sim-to-real transfer strategy is viable: the same controller parameters tuned in Gazebo Harmonic produced stable landings on the Husky rover without requiring gain re-tuning. Residual accuracy degradation in the field is attributed to motion blur, variable illumination and ground-effect turbulence, which are analysed in the limitation and future-work sections that follow.

8.1 Summary of Contributions

The primary contributions of this thesis can be summarised as follows.

1. **Nested multi-scale fiducial target with seamless handoff.** A two-layer ArUco landing target was designed (outer marker $L_{\text{out}} = 0.50\text{ m}$, inner marker $L_{\text{in}} = 0.15\text{ m}$) and paired with an altitude-aware detection logic that switches between the two markers transparently. Unlike fractal marker approaches [2], the proposed scheme uses standard OpenCV ArUco detection at full frame rate (30 Hz), keeping the computational cost compatible with embedded companion computers.
2. **Altitude-scheduled alignment threshold.** A dynamic alignment cone $\varepsilon_{\text{align}}(h) = \varepsilon_0 \cdot \kappa(h)$ was introduced to decouple the descent authorisation from a fixed pixel tolerance. This strategy was shown, both analytically and experimentally, to eliminate the limit-cycle oscillations that arise when a constant threshold is used across a wide altitude range, directly countering the optical-gain-explosion phenomenon described in Chapter 4.
3. **Gain-scheduled PID with feed-forward compensation.** A body-frame velocity controller was designed in which the proportional and derivative gains are attenuated by a damping factor $\gamma(h)$ as altitude decreases, while a feed-forward term injects the Kalman-estimated target velocity into the command. The integral action is progressively disabled below $h = 0.4\text{ m}$ to prevent wind-up in the ground-effect zone. The resulting controller achieved a velocity-tracking $\text{RMSE} = 0.284\text{ m/s}$ in both simulation and outdoor flight.

4. **End-to-end sim-to-real validation with zero gain re-tuning.** The complete software stack—state machine, Kalman filter, PID gains, and feed-forward logic—was deployed from Gazebo Harmonic SITL to a physical Holybro x500 V2 quadrotor without modifying any controller parameter. The outdoor campaign demonstrated stable landings on the Husky rover at 1 km/h under 14 km/h crosswind, confirming that the simulation-based development workflow is a viable strategy for this class of precision-landing problems.

8.2 Limitations

The following limitations have been identified, ordered by their estimated impact on the system’s applicability to operational scenarios.

1. **Maximum tested platform speed.** Real-world validation was limited to 1 km/h by the Husky rover’s maximum safe speed on the available terrain. While the simulation environment allowed preliminary tests at higher speeds, the controller’s behaviour under realistic aerodynamic disturbances at 5–10 km/h remains unverified. This is the most significant constraint on the system’s current Technology Readiness Level, as many operational use cases (e.g., maritime resupply, convoy logistics) require platform speeds of at least 5 km/h. Addressing this limitation will likely require not only a faster test platform but also predictive trajectory planning to compensate for the increased transport delay in the control loop.
2. **Illumination sensitivity.** The ArUco detection pipeline relies on adaptive thresholding, which is sensitive to strong shadows, direct sunlight and low-light conditions. During the

outdoor tests, occasional frame drops were observed when the sun angle produced specular reflections on the marker surface. Although the Kalman filter’s prediction-only mode bridged these gaps without loss of stability, sustained detection failures lasting more than 1.5 s would trigger the lost-target recovery procedure, increasing the total mission time. This limitation is inherent to all binary fiducial markers and represents a fundamental motivation for the learning-based detection approaches discussed in Section 8.3.

3. **Motion blur.** At higher relative velocities or under gusty wind conditions, motion blur degrades the sharpness of the marker corners, increasing the pixel-level localisation noise. The current system has no explicit mechanism to detect or compensate for blur; instead, it relies on the Kalman filter’s measurement noise covariance R to attenuate the resulting jitter. A more principled approach would involve blur-aware corner refinement or adaptive exposure control, as discussed in Section 8.3.
4. **Ground-effect modelling.** Rotor downwash turbulence increases significantly below $h \approx 0.5$ m, introducing vibrations and aerodynamic disturbances that are not captured by the Gazebo physics engine. The gain-scheduling factor $\gamma(h)$ was designed to mitigate this effect empirically, and the outdoor experiments confirmed that the conservative tuning margin is sufficient for the tested conditions. However, a quantitative characterisation of the ground-effect magnitude as a function of altitude, for example, through dedicated thrust-stand measurements or CFD analysis-would allow a more systematic tuning of the damping profile and extend the system’s robustness to heavier platforms with stronger downwash.

5. **Single-axis parallax correction.** The current implementation corrects only for the forward camera offset O_y relative to the UAV centre of mass. A full three-dimensional parallax correction, accounting for both O_y and O_z as well as the instantaneous roll and pitch angles, could improve the touchdown accuracy in the presence of non-zero attitude at contact, particularly at higher platform speeds where the steady-state pitch angle is larger.

8.3 Future Work

Building on the limitations identified above, this section outlines a roadmap of possible extensions, organised into near-term improvements that can be implemented within the existing architecture and longer-term extensions that would require more substantial modifications.

8.3.1 Near-Term Improvements

1. **Higher-speed validation.** The most immediate next step is to test the system on a faster ground vehicle (e.g., a modified RC car at 5–10 km/h) to evaluate the controller and the Kalman filter’s prediction horizon at operationally relevant speeds. This campaign would also reveal whether the current reactive PID architecture is sufficient or whether a predictive planning layer (e.g., model-predictive control) is needed to compensate for the increased transport delay.
2. **Full three-dimensional parallax correction.** Extending the parallax compensation from a single forward offset to a full rigid-body transformation (using the current roll, pitch and yaw from the PX4 estimator) is a straightforward software modification that would improve accuracy at non-zero attitude angles.

3. **Extended Kalman Filter for pixel-to-metre mapping.** Replacing the linear Kalman filter with an Extended Kalman Filter (EKF) that incorporates the nonlinear relationship $\Delta u = f \cdot \Delta d/h$ (Equation 4.2) directly in the measurement model would reduce the estimation lag at very low altitudes, where the linearisation error of the current constant-velocity model becomes significant.

In conclusion, the system presented in this thesis demonstrates that centimetre-level autonomous landing on a slow-moving platform is achievable using only a monocular camera and a passive fiducial target, without requiring external infrastructure or gain re-tuning between simulation and hardware. The modular architecture-nested marker, Kalman filter, gain-scheduled PID, and MAVSDK offboard interface-provides a solid foundation on which the near-term and longer-term extensions outlined above can be incrementally integrated, progressively expanding the system's speed range, environmental robustness and operational readiness.

APPENDICES

APPENDIX A

COPYRIGHT PERMISSIONS

In this appendix, we present the copyright permissions for CC Deed - Attribution 4.0 International, whose contents were used in this thesis.

APPENDIX A (Continued)

4/14/26, 4:26 PM

Deed - Attribution 4.0 International - Creative Commons

English ▼



Canonical URL: <https://creativecommons.org/licenses/by/4.0/>

[See the legal code](#)

You are free to:

Share — copy and redistribute the material in any medium or format for any purpose, even commercially.

<https://creativecommons.org/licenses/by/4.0/>

1/5

APPENDIX A (Continued)

4/14/26, 4:26 PM

Deed - Attribution 4.0 International - Creative Commons

Adapt — remix, transform, and build upon the material for any purpose, even commercially.

The licensor cannot revoke these freedoms as long as you follow the license terms.

Under the following terms:

Attribution — You must give [appropriate credit](#), provide a link to the license, and [indicate if changes were made](#). You may do so in any reasonable manner, but not in any way that suggests the licensor endorses you or your use.

No additional restrictions — You may not apply legal terms or [technological measures](#) that legally restrict others from doing anything the license permits.

Notices:

You do not have to comply with the license for elements of the material in the public domain or where your use is permitted by an applicable [exception or limitation](#).

No warranties are given. The license may not give you all of the permissions necessary for your intended use. For example, other rights such as

APPENDIX A (Continued)

4/14/26, 4:26 PM

Deed - Attribution 4.0 International - Creative Commons

publicity, privacy, or moral rights may limit how you use the material.

Notice

This deed highlights only some of the key features and terms of the actual license. It is not a license and has no legal value. You should carefully review all of the terms and conditions of the actual license before using the licensed material.

Creative Commons is not a law firm and does not provide legal services. Distributing, displaying, or linking to this deed or the license that it summarizes does not create a lawyer-client or any other relationship.

Creative Commons is the nonprofit behind the open licenses and other legal tools that allow creators to share their work. Our legal tools are free to use.

- [Learn more about our work](#)
- [Learn more about CC Licensing](#)
- [Support our work](#)
- [Use the license for your own material.](#)
- [Licenses List](#)

APPENDIX B

COPYRIGHT PERMISSIONS

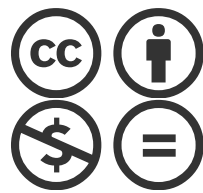
In this appendix, we present the copyright permissions for CC Deed - Attribution-NonCommercial-NoDerivatives 4.0 International, whose contents were used in this thesis.

APPENDIX B (Continued)

4/14/26, 4:57 PM

Deed - Attribution-NonCommercial-NoDerivatives 4.0 International - Creative Commons

English ▼



CC BY-NC- ND 4.0

Attribution- NonCommercial- NoDerivatives 4.0 International

Deed

Canonical URL: <https://creativecommons.org/licenses/by-nc-nd/4.0/>

[See the legal code](#)

APPENDIX B (Continued)

4/14/26, 4:57 PM

Deed - Attribution-NonCommercial-NoDerivatives 4.0 International - Creative Commons

You are free to:

Share — copy and redistribute the material in any medium or format

The licensor cannot revoke these freedoms as long as you follow the license terms.

Under the following terms:

Attribution — You must give [appropriate credit](#), provide a link to the license, and [indicate if changes were made](#). You may do so in any reasonable manner, but not in any way that suggests the licensor endorses you or your use.

NonCommercial — You may not use the material for [commercial purposes](#).

NoDerivatives — If you [remix, transform, or build upon](#) the material, you may not distribute the modified material.

No additional restrictions — You may not apply legal terms or [technological measures](#) that legally restrict others from doing anything the license permits.

APPENDIX B (Continued)

4/14/26, 4:57 PM

Deed - Attribution-NonCommercial-NoDerivatives 4.0 International - Creative Commons

Notices:

You do not have to comply with the license for elements of the material in the public domain or where your use is permitted by an applicable [exception or limitation](#).

No warranties are given. The license may not give you all of the permissions necessary for your intended use. For example, other rights such as [publicity, privacy, or moral rights](#) may limit how you use the material.

Notice

This deed highlights only some of the key features and terms of the actual license. It is not a license and has no legal value. You should carefully review all of the terms and conditions of the actual license before using the licensed material.

Creative Commons is not a law firm and does not provide legal services. Distributing, displaying, or linking to this deed or the license that it summarizes does not create a lawyer-client or any other relationship.

Creative Commons is the nonprofit behind the open licenses and other legal tools that allow creators to share their work. Our legal

CITED LITERATURE

1. Marchi, F.: High-Precision Autonomous Landing Systems for UAVs Using Visual Navigation. Master's thesis, Politecnico di Torino, 2026.
2. Garrido-Jurado, S., Munoz-Salinas, R., Madrid-Cuevas, F. J., and Marin-Jimenez, M. J.: Automatic generation and detection of highly reliable fiducial markers under occlusion. Pattern Recognition, 47(6):2280–2292, 2014.
3. Mahony, R., Kumar, V., and Corke, P.: Multirotor aerial vehicles: Modeling, estimation, and control. IEEE Robotics and Automation Magazine, 19(3):20–32, 2012.
4. Eling, C., Klingbeil, L., and Kuhlmann, H.: Real-time single-frequency gps/lonass/bds rtk positioning for uav applications. Sensors, 15(5):9221–9240, 2015.
5. Alarifi, A., Al-Salman, A., Alsaleh, M., Alnafessah, A., et al.: Ultra wideband indoor positioning technologies: Analysis and recent advances. Sensors, 16(5):707, 2016.
6. Russo, L., Mondal, M. S., Ramasamy, S., Humann, J. D., Dotterweich, J. M., and Bhounsule, P. A.: Precision auto-landing of an aerial vehicle on a moving ground vehicle: A modular ros2 approach. In Proceedings of the ASME International Design Engineering Technical Conferences and Computers and Information in Engineering Conference (IDETC/CIE 2025), Volume 5: 21st IEEE/ASME International Conference on Mechatronic and Embedded Systems and Applications (MESA); 49th Mechanisms and Robotics Conference (MR), page V005T08A036, August 2025.
7. Austin, R.: Unmanned Aircraft Systems: UAVs Design, Development and Deployment. Chichester, UK, John Wiley & Sons, 2010.
8. Hérissé, B., Hamel, T., Mahony, R., and Russotto, F.-X.: Landing a VTOL unmanned aerial vehicle on a moving platform using optical flow. IEEE Transactions on Robotics, 28(1):77–89, 2012.
9. PX4 Autopilot: PX4 flight stack documentation. <https://docs.px4.io>, 2026. Accessed February 2026.

10. Rigogliuso, L.: Robust Autonomous Landing of a UAV on a High-Speed Platform. Master's thesis, Politecnico di Torino, Turin, Italy, December 2022.
11. Lee, D., Ryan, T., and Kim, H. J.: Autonomous landing of a VTOL UAV on a moving platform using image-based visual servoing. In Proceedings of the IEEE International Conference on Robotics and Automation (ICRA), pages 971–976, 2012.
12. Kyristis, S., Antonopoulos, A., Chanielakis, T., Stefanakis, E., Linardos, C., Tripolitsiotis, A., and Partsinevelos, P.: Towards autonomous modular UAV missions: The detection, geo-location and landing challenge. In Proceedings of the IEEE International Conference on Unmanned Aircraft Systems (ICUAS), pages 661–670, 2016.
13. Araar, O. and Aouf, N.: A new hybrid approach for the visual servoing of VTOL UAVs from unknown surfaces. IEEE/ASME Transactions on Mechatronics, 22(3):1042–1051, 2017.
14. Falanga, D., Zanchettin, A., Simovic, A., Delmerico, J., and Scaramuzza, D.: Vision-based autonomous quadrotor landing on a moving platform. In Proceedings of the IEEE International Symposium on Safety, Security and Rescue Robotics (SSRR), pages 200–207, 2017.
15. Ling, K., Chow, D., Das, A., and Bhatt, T.: Autonomous maritime landings for low-cost VTOL aerial vehicles. In Proceedings of the IEEE Conference on Control Technology and Applications, pages 32–39, 2014.
16. Garrido-Jurado, S., Muñoz-Salinas, R., Madrid-Cuevas, F. J., and Medina-Carnicer, R.: Generation of fiducial marker dictionaries using mixed integer linear programming. Pattern Recognition, 51:481–491, 2016.
17. Polvara, R., Patacchiola, M., Sharma, S., Wan, J., Manning, A., Sutton, R., and Cangelosi, A.: Toward end-to-end control for UAV autonomous landing via deep reinforcement learning. In Proceedings of the IEEE International Conference on Unmanned Aircraft Systems (ICUAS), pages 115–123, 2018.
18. Rodriguez-Ramos, A., Sampedro, C., Bavle, H., de la Puente, P., and Campoy, P.: A deep reinforcement learning strategy for UAV autonomous landing on a moving platform. Journal of Intelligent & Robotic Systems, 93:351–366, 2019.
19. Dou, J., Xu, B., and Dou, L.: Robust gnss positioning using unbiased finite impulse response filter. Remote Sensing, 15:4528, 09 2023.

20. Huang, S., Guo, y., Zha, S., Wang, F., and Fang, W.: A real-time location system based on rfid and uwb for digital manufacturing workshop. Procedia CIRP, 63:132–137, 12 2017.
21. Badakis, G., Koutsoubelias, M., and Lalis, S.: Robust precision landing for autonomous drones combining vision-based and infrared sensors. In Proceedings of the IEEE Sensors Applications Symposium (SAS), 2021.
22. Olson, E.: Apriltag: A robust and flexible visual fiducial system. In Proceedings of the IEEE International Conference on Robotics and Automation (ICRA), 2011.
23. Bradski, G.: The OpenCV library. Dr. Dobb’s Journal of Software Tools, 2000.
24. Lv, M., Fan, B., Fang, J., and Wang, J.: Autonomous landing of quadrotor unmanned aerial vehicles based on multi-level marker and linear active disturbance rejection control. Sensors, 24(5):1645, 2024.
25. MAVSDK Project: MAVSDK developer guide. <https://mavsdk.mavlink.io>, 2026. Accessed February 2026.
26. Koenig, N. and Howard, A.: Design and use paradigms for Gazebo, an open-source multi-robot simulator. In Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2004.
27. Gazebo Harmonic: Gazebo simulator documentation. <https://gazebo.org>, 2026. Accessed February 2026.
28. Kalman, R. E.: A new approach to linear filtering and prediction problems. Transactions of the ASME – Journal of Basic Engineering, 82:35–45, 1960.
29. Tobin, J., Fong, R., Ray, A., Schneider, J., Zaremba, W., and Abbeel, P.: Domain randomization for transferring deep neural networks from simulation to the real world. In Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), pages 23–30, 2017.
30. Holybro: X500 V2 development kit. <https://holybro.com/products/x500-v2-kit>, 2024. Accessed January 2026.
31. Clearpath Robotics: Husky unmanned ground vehicle. <https://clearpathrobotics.com/husky-unmanned-ground-vehicle-robot/>, 2024. Accessed January 2026.

VITA

FRANCESCO MARCHI

EDUCATION	B.Sc., Management Engineering, Università degli studi di Padova, Italy, 2024 M.Sc., Electrical & Computer Engineering, UIC, Chicago, 2026 M.Sc., Mechatronic Engineering, Politecnico di Torino, Italy, 2026
EXPERIENCE	Football Referee Mon. 2020 – 2024
LANGUAGE SKILLS	Italian Native Speaker English C2
AWARDS	Italian Scholarship for International Studies
MEMBERSHIPS	IEEE - Institute of Electrical and Electronics Engineer - Student Membership